Classify by Position, Not by Label: LLM-Driven Extraction of Hierarchical Structure from Fragmented US Early Learning Standards

Emily Cheyne
Founder, EdTech Co.
emily@edtechco.org

Abstract

Every US state publishes early learning standards, the developmental expectations for children from birth to kindergarten, and each designs its own document, so software cannot read this public policy. The obstacle is not text recovery but hierarchy recovery: a standard is meaningful only through its ancestry, and the surface cues that reveal ancestry are exactly what varies. Arizona writes `Strand 1:`, Kentucky writes `Benchmark 1.1`, Nevada's code namespace skips a level, and Colorado has no sub-strand tier at all.

I present an LLM-driven method that classifies each element by its *nesting position* rather than its label vocabulary. A first pass infers a per-document depth map and a second classifies every element against it; a decoupled parser resolves the result into one canonical schema with deterministic identifiers, and a serverless batching architecture applies it to whole documents. There is no per-state code path, though the prompts carry worked examples from the corpus (§5.7).

On an annotated subset of four development states and two validation states, detector recall is 1.000 at every hierarchy level in all six, and a manual audit of all 297 in-scope detections finds one hallucinated element. Ablating the depth map leaves domain and indicator recall untouched while strand falls to 0.920 and sub-strand to 0.833, exactly the levels whose identity is positional, and a rule-based extractor matches the method at the top division but collapses beneath it. Repeated runs show the pipeline is not deterministic, with instability concentrated in three known places, and one of my own earlier results failed to reproduce; I report it as a non-reproduction. That reporting discipline is offered as a contribution: every table names its corpus tier, every unmatched detection is audited by hand, and every tabulated number is regenerated from a recorded artifact.

1 Introduction

Every US state publishes early learning standards: the developmental expectations that shape curriculum, assessment, and teacher preparation for children from birth through kindergarten entry. More than fifty jurisdictions issue them independently (National Center on Early Childhood Quality Assurance, n.d.; Education Commission of the States, 2024), and each designs its own document. The result is a body of policy that is public, consequential, and almost entirely unavailable to software. Unlike K–12 academic standards, which have machine-readable distribution formats (Sutton and Golder, 2008; 1EdTech Consortium, 2017), early learning standards exist largely as PDFs: multi-column tables, age-banded grids, and prose narratives whose typography is shared by no two states.

The obstacle is not optical character recognition. It is that a standards document is a *tree* rendered as a page. A single indicator is meaningful only through its ancestry: which domain, which strand, which sub-strand it sits under, and at what age band. Recovering that tree requires deciding, for every line, how deeply it is nested, and the surface cues that would answer the question are exactly what varies between states. Arizona writes `Strand 1: Language and Literacy`; Kentucky writes `Benchmark 1.1`; Nevada prints a code namespace that skips a level entirely; Colorado publishes no sub-strand tier at all. A rule tuned to one document's vocabulary is, by construction, a rule that fails on the next.

My central claim is that a language model can recover this structure by classifying each element's **nesting position** rather than its document-specific label, and that doing so is what generalizes. Concretely, I first infer a per-document *depth map* (how many nesting levels this particular document uses, and what each looks like) and then classify every

element against that map as an authority, rather than against a fixed vocabulary of level names. The label `Benchmark 1.1` then does not have to be recognized as a sub-strand; it has to be recognized as sitting one level below whatever `Language and Early Literacy Standard 1` is.

This framing also determines how the system is built. I hold a deliberate design discipline that structural decisions belong in the prompt as general document principles, not in Python as per-document branches. That discipline was adopted after an earlier rule-driven revision scored well on the states it had been developed against and failed to transfer; the migration that removed those rules is described in §5.7. Deterministic code is retained only where it reads the *shape* of an output (dot-segment structure, whitespace, string containment) and never any document's words.

The absence of a labeled corpus is what rules out the obvious supervised alternative. There is no annotated dataset of hierarchically-parsed early learning standards at any scale, so a fine-tuned model would first require one to be built, and the dominant cost of that path is annotation rather than compute. My own goldens took a high level of effort to produce and cover six states; they are an evaluation instrument, not a training set. An in-context approach that needs no labeled data is therefore not merely cheaper here, it is the only approach available without first solving a different problem (cf. Liu et al., 2022 for the counterargument where labels do exist).

Contributions.

1. A two-stage detection method built on per-document depth-map inference and classify-by-nesting-position prompting, with no per-state code path (§5.2; what the prompts do carry is itemized in §5.7).

2. A parsing stage that resolves detected elements into a single canonical schema across documents that do not agree on level count or code namespace, producing deterministic identifiers whose uniqueness is enforced by a resolver, and reporting how often that resolver has to act rather than assuming it never does (§5.3).

3. A serverless architecture that batches detection and parsing through a prepare–map–merge pattern so the method applies to whole standards documents rather than excerpts (§5.6, Appendix F).

4. An evaluation over four development states and two validation states (§5.7), at an annotated subset tier of each document. Detector recall is 1.000 at every hierarchy level in all six states, and a manual audit of all 297 in-scope detections at that tier finds a single hallucinated element (verified precision 0.997). An ablation shows the depth-map pass is what carries the intermediate levels: removing it leaves domain and indicator recall untouched while strand falls to 0.920 and sub-strand to 0.833 (§6.2).

5. A reporting discipline that the rest of the evaluation is built to satisfy, and which I offer as a contribution rather than as a caveat: every table is labelled with the corpus tier it was computed at; every unmatched detection is audited by hand and signed rather than assumed to be a false positive; every tabulated number is regenerated from a recorded artifact by a named script; a validation guard is shown catching a live defect rather than asserted to work; and a previously recorded result of my own that failed to reproduce is reported as a non-reproduction instead of quietly replaced (§6.6, §7).

Scope of the reported numbers. Every headline quality measurement in this paper is computed at the *subset* corpus tier: hand-trimmed excerpts of one to two domains per state, 70 pages in total, fully annotated. This is favorable preprocessing and I state it plainly rather than leaving it to be inferred; §3 gives the tiers and retained page ranges, and §7 discusses what does and does not transfer to whole documents. Cost, scale and one quality measurement are reported at the *trimmed* tier instead: a document's standards content in full, with front matter, expository essays and appendices removed. The tiers are never mixed within a table, and each table names its own (§6.7, §6.8).

2 Related Work

Hierarchy recovery from documents. Logical-structure recovery has a long lineage: metadata pipelines, page-object datasets, layout-aware encoders, and OCR-free transcribers (Lopez, 2009; Zhong et al., 2019; Xu et al., 2020; Huang et al., 2022; Kim et al., 2022; Blecher et al., 2023). None

of them targets a typed tree carrying codes and ancestry. Work that does reconstruct the tree parses renderings, or decomposes the task into page-object detection, reading order and tree construction (Rausch et al., 2021; Ma et al., 2023; Wang et al., 2024, 2025; Xing et al., 2024). That line shares a premise my setting violates: hierarchy can be *learned*, because supervision exists over a genre whose typography is internally consistent. Where every jurisdiction designs its own document (National Center on Early Childhood Quality Assurance, n.d.) there is neither. Two training-free neighbors come closer. HiPS (Wehnert et al., 2025) segments law books from table-of-contents metadata, falling back to typographic cues refined by an LLM, within one genre; DocsRay (Jeong et al., 2025) prompts for a pseudo table of contents, but targets retrieval segmentation rather than a typed hierarchy graded against structural annotations. What these documents share is not appearance but nesting, so my depth-map pass is functionally the table-of-contents subtask of Wang et al. (2024), inferred zero-shot from content and then treated as the authority every element is classified against.

Extraction without a labeled corpus. Generative extraction is competitive across classic IE tasks, especially where labeled corpora are scarce (Wadhwa et al., 2023; Xu et al., 2024; Agrawal et al., 2022), though where labels do exist, parameter-efficient fine-tuning can beat in-context learning (Liu et al., 2022). GoLLIE (Sainz et al., 2024) is the closest analogue: training a model to follow annotation *guidelines* makes it generalize to unseen schemas. My prompts do this without fine-tuning, stating document-agnostic structural principles a frozen model applies to unseen states. That is the available move rather than the cheap one, since no annotated corpus of parsed early learning standards exists. Three choices follow: schema-validated JSON with deterministic repair rather than constrained decoding (Willard and Louf, 2023); domain-scoped chunking rather than whole-document prompting, given positional degradation in long contexts (Liu et al., 2024); and reported run-to-run stability (Wang et al., 2023) rather than a single run. A generative extractor can emit content that is not in its source (Ji et al., 2023); I measure that directly, by auditing every unmatched detection by hand rather than assuming it is a false positive. Structural consistency is enforced separately, at the validation boundary, where an indicator's code must extend its ancestor's; that check is the analogue of the totals-at-each-level validation Aggarwal et al. (2025) apply to fiscal tables. Absent labels, human effort goes to verification rather than annotation: every record is reviewed at all four levels, in the manner of machine teaching rather than active learning, uncertainty-guided allocation, or the model as annotator (Mosqueira-Rey et al., 2023; Monarch, 2021; Settles, 2009; Li et al., 2023; Gilardi et al., 2023; Tan et al., 2024). Review is blanket; confidence gates nothing.

Machine-readable education standards. Two decades of work assume a normalized set of standards already exists. ASN modeled US K–12 standards in RDF (Sutton and Golder, 2008) and CASE defines an exchange format (1EdTech Consortium, 2017), but ASN was populated by hand and CASE is a format, not an extractor; downstream work measures alignment and assigns resources and items *to* standards already normalized (Porter, 2002; Polikoff et al., 2011; Devaul et al., 2011; Xu et al., 2025). For early childhood the assumption fails outright: the one federal framework for the age band binds Head Start programs alone (Office of Head Start, 2015), and each state publishes its own PDF (National Center on Early Childhood Quality Assurance, n.d.; Education Commission of the States, 2024). The step this paper automates is not a competitor to that literature but its missing input.

3 Corpus

The corpus is the early learning standards documents of six US states: four used for development (Arizona, California, Colorado, Texas) and two used for validation (Nevada, Kentucky). No code path names the validation states, but their failures did prompt general fixes, so they are not a clean held-out test (§5.7). Table 1 summarizes what the annotated portion of each contains; Appendix E names each source document and lists the pages retained at each tier.

Each document exists at up to three tiers, and the two reductions are not the same kind of reduction. The *full* tier is the published PDF as issued. The *trimmed* tier removes non-standards matter (front matter, introductions, guiding-principles essays, appendices, acknowledgements) and **retains every standard**; it reduces page count, and therefore cost, but not coverage, and it is the tier the scale measurements of §6.7 and §6.8 run on. Kentucky's 52 trimmed pages are all of Kentucky's standards.

The *subset* tier is a genuine reduction in coverage: one to two domains per state, 8–15 pages each and 70 in total, chosen to be cheap to iterate on and to annotate exhaustively. **Every headline quality number in this paper is computed at this tier**; the one quality measurement at the trimmed tier (§6.8) is kept in its own table. Both cuts were made by hand, which is favorable preprocessing; Table 11 lists the retained pages so that either tier can be reconstructed from the issuing agency's copy. It gives two numberings, because they are not the same number: a page's index in the PDF file, and the page number printed on the page, which differ by each document's unnumbered front matter. Kentucky's PDF page 52 is printed page 39, and its subset also contains PDF page 65, whose printed number is 52, so "page 52" is ambiguous on that document alone. Printed numbers are recovered automatically and are reported only for the two documents where that recovery is reliable across the whole tier.

Three sources need naming precisely. California's document is the state's Preschool/Transitional Kindergarten Learning Foundations "at a glance" publication (68 pages), which was cut directly to its 13-page subset with no trimmed tier. Colorado's is the 41-page Ages 3–5 volume of the state's birth-to-eight guidelines; its trimmed tier is the whole document, so nothing was cut from it, and its out-of-scope content is other age bands rather than standards removed by trimming. Nevada's document is its 2023 pre-kindergarten standards, a multi-domain document.

Two properties of Table 1 are results rather than document facts. Its age-band column counts the distinct bands a state uses, not coverage: every one of the 262 standards carries an age band. And the identifier resolver never had to separate two standards in any state in any logged subset-tier run, which matters because the identifier is the key under which a standard is stored (§4). The table's collision column is counted after that resolver and so is zero by construction; the evidence that it had nothing to resolve is given in §6.1.

4 Canonical Schema

The target of the pipeline is a single schema that every state's document is resolved into, regardless of how that document organizes itself. It defines four hierarchy levels, *domain*, *strand*, *sub-strand*, and *indicator*, of which strand and sub-strand are optional. Each level carries a code, a name, and an optional free-text description; the indicator additionally carries an age band, a source page, and the verbatim source text it was extracted from. A normalized standard is one indicator together with its full resolved ancestry, so the unit of storage is a leaf-with-lineage rather than a node in a tree.

Two properties of this schema do real work and are worth stating explicitly.

Optionality is not decoration. A four-level schema is not a four-level document. Colorado publishes no sub-strand tier, and Texas realizes almost none; Nevada uses all four but prints a code namespace that skips the strand entirely, coding indicators as `{domain}.{sub_strand}.PKn`. Requiring every level to be populated would force the pipeline to invent structure, which is the specific failure the depth-map ablation induces when it is disabled (§6.2). The schema instead records how many levels a document actually uses, and the evaluation grades that decision rather than assuming it.

Identifiers are deterministic and self-contained. Every standard receives an identifier of the form

```
{country}-{state}-{year}-
{indicator_code}
```

where the indicator code is already fully qualified: it carries its own ancestry, and it carries whatever disambiguator distinguishes side-by-side variants of the same outcome. Texas prints the same expectation once per age column, so the age token lives inside the code (`PK3.I.A.2` against `PK4.I.A.2`); California, for some standards, prints the same foundation once per proficiency column, so the column token does (`ELD.1.0.VOCA.1.1.DISC`). Detection is told to emit one element per column rather than one element with several descriptions (Appendix A.2.3), and parsing is told which suffix each kind of column takes (Appendix A.3.4). There is no separate domain component in the identifier and no age band appended after the fact: the derivation is one rule over an already-qualified code, which is what makes the identifier reproducible from the document rather than from the order in which the pipeline happened to process it. Two standards can still compose the same code, so uniqueness is enforced by a resolver after parsing (§5.3). A collision count taken after that resolver is zero by construction, so what I report is how often the resolver had to act (§6.1, §6.8).

State	Role	Subset pp	Full pp	Dom.	Str.	Sub.	Standards	Age bands	ID coll.
AZ	dev	15	217	2	4	9	45	1	0
CA	dev	13	68	3	7	18	94	3	0
CO	dev	10	41†	3	10	0	48	1	0
TX	dev	9	87	2	4	2	25	2	0
NV	validation	15	98	3	7	7	24	1	0
KY	validation	8	120	3	5	10	26	1	0
Total		70		16	37	46	262	6	**0**

Table 1: Corpus descriptive statistics. Domain, strand and sub-strand columns count *distinct* nodes in at least one standard's ancestor chain, so they are smaller than the detector's element counts in Table 2. **Age bands** is the state's distinct bands; its total is the union across states, not a column sum. **ID coll.** is counted after the identifier resolver and is zero by construction (§6.1). CO's subsets derive from the 41pp ages-3–5 document (†). All headline quality numbers in this paper are measured on the *_only_subset* tier shown here (§3); the one quality measurement at the *_trimmed* tier is Table 8. Source: `task8_20260904`.

Building a canonical schema across independently-formatted jurisdictions is itself a nontrivial part of the contribution, because the states do not agree on the two things a naive schema would take for granted: how many levels there are, and what a code means. Nevada's namespace skipping a level and Kentucky's dotted sub-level identifiers, which restate their parent's position inside the child's own code, are not exceptions to be special-cased. They are the normal condition of the corpus, and §5.3 describes the shape-based rules that resolve them without reference to any document's vocabulary.

Finally, the schema carries per-element human verification state (`human_verified`, `verified_at`, `verified_by`) at all four levels, supporting an editorial review workflow over the extracted corpus. This is a separate concept from the model's confidence score, and the two are never combined; §5.5 explains why. The full schema is given in Appendix B and annotated examples in Appendix C.

5 Method

The system is a six-stage pipeline. **Ingestion** registers a source PDF and its provenance. **Extraction** recovers text blocks with their page numbers and page coordinates, using a hybrid of OCR and the PDF's own text layer, since either alone is unreliable on the multi-column and tabular layouts these documents favor. **Detection** is where structure is recovered, emitting a flat list of typed elements, each carrying a level, a code, a title, an optional description, an age band, and the source text it was read from. **Parsing** resolves that flat list into the canonical schema of §4, attaching each indicator to its full ancestry and composing its identifier. **Validation** checks each record against the schema and against structural invariants on the identifier, and **persistence** writes the surviving records to a relational store. The two stages that recover structure are the subject of this section.

5.1 Two-Stage Design Rationale

Recovering a standards hierarchy involves two decisions that are easy to conflate and better kept apart: *what is on the page*, and *what it is a child of*. The first is local (a heading, a numbered item, a paragraph of prose) and can be judged from the text in front of the reader. The second is global: an element's parent may be several pages back, may be printed only once at the top of a section that runs for multiple pages, or may not be printed at all and must be recovered from the codes of its own descendants. Detection is accordingly forbidden from reconstructing an absent parent out of the position identifiers inside a child's code, and emits the children alone (Appendix A.2.2); recovering that parent is parsing's work, not detection's.

I therefore split the work. **Detection** reads the document in overlapping chunks and emits a flat list of typed elements, each labeled with its hierarchy level but not linked to a parent. **Parsing** takes that flat list, groups it by domain, and resolves each indicator's full ancestry and identifier. The split buys three things. It lets each stage see the context it actually needs, so parsing reasons over an entire domain's elements at once rather than over one chunk's worth of text. It lets the two stages be graded independently against separately annotated goldens, which is what makes a detection failure distinguishable from a resolution failure. And it lets the two stages run on different models, since they are not equally hard (§5.4).

5.2 Detection

Detection is two passes over the extracted text blocks.

Pass 1: depth-map inference. Before any element is classified, a single cheap model call infers the document's *depth map*: how many nesting levels this document uses, and what an element at each level looks like in it. This is the paper's central move. Rather than asking the model to recognize "strand" or "benchmark" as level names, vocabulary that no two states share, I ask it once, per document, to characterize that document's own nesting scheme, and then treat the answer as authority for every subsequent classification. Appendix A.1 gives that prompt in full; it is short, and it is the only one reproduced whole.

The pass runs on a sample of blocks rather than the whole document, under a 6,000-token budget. This protocol is significant because every element's level is assigned against the map, *a level missing from the sample is a level missing from the entire run*: if the sample contains no evidence that a document nests four deep, the map reports three, and the elements shift accordingly.

The obvious sampler, keeping every nth block, is the wrong one here. It keeps each line with the same probability $1/n$ however much structural evidence that line carries, and in a standards document evidence and frequency run opposite to one another: a section heading appears once per section, while the body prose beneath it is repeated on every page. Uniform sampling therefore drops precisely the headings that establish the top of the hierarchy and keeps the prose, which establishes nothing, on volume alone.

I stratify by layout instead. Extraction already records each block's normalized left edge, and that edge is already part of what the model is shown; since indentation tracks nesting, a rare horizontal position *is* a rare depth. Blocks are therefore bucketed by left edge, and each bucket is guaranteed a floor of twelve blocks spread across the document. Buckets are filled rarest-first, so when the budget binds, what it truncates is the bulk of the document rather than the structurally distinctive levels. The guarantee is cheap for the same reason it is needed: those levels are rare, and in Kentucky's complete standards document the heading buckets together hold only a small fraction of all blocks.[1] Blocks whose page geometry is unusable have no left edge to group on, and fall back to a coarse text-shape key reading word count and final character only. Either way the rule reads layout position and counts, never any document's words.

Pass 2: classification against the map. The document is then chunked into roughly 2,000-token windows with 500 tokens of overlap, and each chunk is classified with the depth map supplied as context. The overlap exists because elements straddle chunk boundaries; because chunks overlap, a long description split across a boundary can be reconstructed by splicing the shared span, which recovers passages that neither chunk holds whole.

The prompt states general document-structure principles rather than per-state rules. Its central instruction is positional: classify an element by where it sits in the depth map, not by the word its label happens to use (Appendix A.2.1). Supporting rules are stated at the same level of generality: a heading of the form `<Label> <id>: <Title>` has the label-and-id as its code and the text after the colon as its title, for *any* structural label word rather than a fixed list; a code, when the document prints one, is authoritative and may appear inline, in a caption beside the heading, or as the shared leading prefix of its descendants' codes; and when the document prints none, a code is derived from the title by a stated abbreviation procedure capped at five characters. All three are one rule, reproduced at Appendix A.2.4.

Where determinism is enforced in code. The last of those rules, deriving a code from the element's title when the document prints none, is different in kind from the others. The rest ask the model for a judgement about a document; this one asks it to execute a deterministic string algorithm, and a sampling model executes such an algorithm inconsistently. In an engineering measurement on Kentucky (three runs at temperature zero against one frozen extraction, recorded in the project's design notes rather than as a paper artifact) roughly a quarter of the elements received a different code on at least one run, while level, source page, and age band never varied. Because the code becomes part of a database primary key, a rate near zero is not good enough. The abbreviation is therefore also executed in Python, but only where the document supplied no code of its own, decided by two independent guards: a *shape* guard, since the abbreviation branch emits uppercase letters and nothing

[1] An observation from the project's engineering notes rather than a recorded experiment.

else, so a code carrying a digit, separator, or lowercase letter is positional and is never touched; and a *grounding* guard, since a code read off the page appears in the element's own source text while an invented abbreviation does not. Both are required: a model that transcribes a lettered item without its list prefix defeats grounding alone.

This is a pairing, not a substitution. The rule stays in the prompt verbatim; the code makes its floor zero. The same pattern recurs throughout the system, and §5.7 states the discipline that governs it.

5.3 Parsing

Parsing consumes the flat element list, batched by domain so that an entire domain's elements are resolved together, and produces normalized standards.

The substantive work is composing each indicator's fully-qualified code from its chain of ancestors, and the difficulty is that printed code namespaces are not uniform. Three shapes recur. A namespace may *skip* a level, as Nevada's does: its sub-strand code does not extend its strand code, and requiring it to would corrupt every one of the 24 annotated Nevada standards, all of which break the chain at exactly that level. A sub-level identifier may be *dotted*, restating its parent's position inside its own id, as Kentucky's `Benchmark 1.1` does, whose leading `1` *is* Standard 1; so composing it onto the full parent code counts that parent twice. And a heading may print a label-and-id form that must be converted into the domain-qualified form before it can be composed at all (Appendix A.3.3).

Each is handled by a rule keyed on structure rather than vocabulary. Peeling an ancestor's code from a descendant's stops where the namespace stops: if a level's anchored code would equal the code of the level directly below it, the peel has run past the end of the namespace and that level keeps its own heading's identifier (Appendix A.3.2). A code that already carries its own qualification is used as it stands and is never qualified a second time, at any level (Appendix A.3.1). A dotted child whose added segment repeats its parent's last segment is collapsed, but only when the added tail is itself dotted, since a single added segment is an ordinary child index. A repair that cannot demonstrably improve the row's own ancestry is not made, and a repair that would introduce whitespace into an identifier declines rather than trading one malformation for another. None of these rules contains a list of label words; whitespace and dot structure are the only signals they read.

Uniqueness is enforced after the merge rather than during it, because a collision can span batches. Colliding standards are re-qualified using their own ancestors' segments, and *every* member of a colliding set is rewritten, including the first one seen, so the result does not depend on the order in which chunks arrived. Where two standards share every ancestor and remain indistinguishable, a numeric suffix is assigned in document order. That is reproducible, but it carries no information, which is why it is a last resort and is logged as one.

Finally, a validation stage rejects any record whose codes are structurally impossible before it reaches the database: no code may contain whitespace, the indicator's code must extend its nearest present ancestor's, and the identifier must end with the indicator code. The middle condition is deliberately scoped to the leaf, because, as Nevada shows, an intermediate level may legitimately break the chain.

5.4 Model Assignment

The three model calls in the pipeline are not equally difficult, and I assign the cheapest model that suffices for each. The depth-map pass produces a short structural summary from a sampled fraction of the document and runs on Claude Haiku 4.5. Detection is the hard classification problem and runs on Claude Opus 4.6. Parsing operates on already-typed elements with the ancestry constraints supplied explicitly, and runs on Claude Sonnet 4.6. Since detection is the only stage invoked once per chunk of the whole document, this assignment also concentrates spend where the difficulty is. I state the principle here as a design commitment; §6 reports what the assignment costs, and a model-tier ablation is identified as future work.

5.5 Confidence and Human Verification

Every detected element carries a confidence score, and **nothing in the pipeline thresholds it**. There is no review gate, no minimum score, and no field marking an element as requiring attention; all elements flow through detection, parsing, validation, and persistence regardless of score. The prompt does ask for a band, and Appendix A.2.5 reproduces what it asks for; nothing downstream reads the answer. I am explicit about this because it is easy to assume otherwise, and because my own measurement of the score's distribution argues

against building such a gate: the scores occupy five distinct values in $[0.90, 0.97]$, the band the prompt describes as indicating uncertainty is used zero times, and the score varies more by document than by whether an element is correct (§6.5).

Human verification is a separate mechanism. The schema carries per-element verification state at all four levels, and an editorial interface lets a reviewer confirm or correct each element against the source PDF. It is a workflow over the stored corpus, not a filter applied during extraction, and it is blanket rather than score-triggered.

Whether every standard must be verified before the corpus is useful is therefore a cost-and-trust decision rather than a precondition. The durable win is that fifty independently-formatted documents become one queryable corpus, and that value is available before every leaf has been signed off, provided the verification state travels with the data so a consumer can decide what it requires: uses carrying regulatory or assessment weight will reasonably demand verified records, exploratory ones reasonably will not.

5.6 Deployment

The method is deployed as a serverless pipeline whose stages communicate by writing JSON to S3 rather than by passing payloads directly, so each stage's input and output can be inspected in isolation when a run is debugged. Detection and parsing are batched through a prepare → `Map` → merge pattern with at most three concurrent workers, so that a long document does not exhaust a function timeout; detection batches by text-block chunk and parsing by domain, which keeps related elements in the same call. Appendix F gives the deployed architecture and the interface through which the extracted corpus is reviewed.

The evaluation suites exercise the direct path rather than this batched one, and the two are known to diverge (§7). Every subset-tier quality number in this paper is therefore a measurement of the direct path; §6.7 and §6.8 are the evidence that the batched path behaves as intended.

5.7 LLM-First Design Discipline

I made an explicit engineering decision about where structural logic may live: *in the prompt, as a general document principle; not in Python, as a per-document branch.*

This was adopted in response to a measured failure. An earlier revision achieved strong scores on the states it had been developed against by accumulating targeted logic for each: regular expressions for particular label words, a special case for one state's collision behavior, an abbreviation helper tuned to another's conventions. As a result, when tested against a state it had not been developed against, the pipeline performed poorly. A migration removed that logic and re-expressed each rule as a general principle in the prompt: instead of a pattern matching `Strand N:` or `Concept N:`, a rule about what a `<Label> <id>: <Title>` heading means for any label word; instead of a per-state routine stripping a trailing structural noun from domain titles, a rule that such a noun is not part of the name.

Deterministic code is retained, but only where it earns its place by reading the *shape* of an output rather than any document's vocabulary: JSON extraction and schema validation; identifier derivation from already-clean fields; cross-chunk reconciliation of codes that disagree between chunks; the canonicalization of absence to a single spelling; the layout-stratified sampler; the code repairs of §5.3. Each is paired with a prompt rule rather than substituted for one, and each exists because a model sampling at temperature zero emits both spellings of some decision intermittently. A prompt rule lowers that rate but cannot drive it to zero, and a primary key needs zero.

I also decided to enforce the discipline in the development process, not just state it. A new per-state pattern in the detector or parser is treated as a regression even when it raises a score on the development states, and every change is validated against the two validation states before it is accepted (§5.7 describes what that does to their results). The rule-based comparison of §6.3 is built deliberately as a throwaway outside the pipeline for the same reason: it is the concrete form of the approach this discipline forbids, kept so the difference can be measured rather than asserted.

What the prompts do carry. The discipline above is a claim about *code paths*, and it should not be read as a claim that the prompts are free of the corpus. They are not. Both prompts state their rules generally, for any structural label word and at any nesting depth, but each rule is illustrated with worked examples, and those examples were written while looking at real documents. Checking every annotated golden title of at least twelve characters against the verbatim prompt source finds fifteen

that are distinctive to a single state's golden, plus one domain name that several states share and that therefore carries no provenance.[2] Ten of the distinctive strings come from the four development states, which is ordinary few-shot practice.

Five do not, and I state that plainly because it qualifies how the validation states should be read. Four Kentucky strings and one Nevada string appear in the prompts, and two of the Kentucky ones are long indicator sentences ("Engages in an activity for a sustained period of time" → `EASPT`) rather than generic vocabulary. They illustrate the abbreviation procedure of §5.2, which is a deterministic string algorithm over a title and reads no document's meaning; nothing about a level's classification, a code's namespace or a document's depth was written from those states. It is nonetheless one reason the Kentucky and Nevada rows are validation results rather than held-out ones. The larger reason follows.

How the validation states were used. Nevada and Kentucky are validation states, not untouched test states. No code path names either of them, but neither had a golden until partway through development. Once they did, every change was checked against them before it was accepted (§5.7), and several of the general rules in this section were written in response to failures they exposed. Those are the rule that a code may be read from a caption or from the shared prefix of its descendants' codes (Nevada's domain and sub-strand codes), the rule that peeling stops where the namespace stops (Nevada's skipped level), the layout-stratified Pass-1 sampler (Kentucky's complete document, which had been reported as three levels instead of four), and the conversion of a label-form parent code (Kentucky's parser, which recorded 17 of 26 fully correct standards on one run before that rule existed and 26 of 26 after it, the figure in Table 3). Each was stated as a shape rule rather than a state rule and checked against the development states' goldens. They are still fixes prompted by failures on these two documents, so the rows for those documents in §6.1 are validation results. A state examined only after development was frozen is the measurement this paper does not have.

[2] Regenerated by `paper/analysis/prompt_example_provenance.py`; record at `paper/results/task12_20260905/`.

6 Experiments and Results

6.1 Headline Quality: Development and Validation States

Tables 2 and 3 report the two suites on all six states, four developed against and two used for validation (§5.7 describes what that role did and did not include). All numbers are on the `_only_subset` corpus tier (§3), never on full documents.

The precision column needs its definition stated. *Verified precision* is (in-scope detections − hallucinations) divided by in-scope detections, taken from the manual false-positive audit: every unmatched in-scope detection in all six states was examined and the verdict signed by the author. I deliberately do not report the suites' own raw precision here. The detector goldens are partial spot-checks rather than exhaustive annotations, so an unmatched detection is far more often a gap in the annotation than an error by the model, and raw precision therefore measures annotation coverage rather than correctness (§7). Kentucky is the single exception, being the one state whose detector golden is exhaustive.

Recall is 1.000 at every level in all six states, on both the development and the validation pair. The parser's collision column is zero by construction, because the identifier resolver of §5.3 renames any collision before it is counted. The informative fact is that the resolver found nothing to rename: it logs every time it acts, and in the 29 parser runs of §6.6 whose logs were retained (same tier, same code version) it never did. The per-state code and description figures are single-run values and should be read with §6.6 in view: Nevada's code accuracy in particular is properly a range rather than the point value a single recording suggests.

6.2 Ablation: Does the Depth-Map Pass Do Real Work?

The claim this paper leads with is that hierarchy should be recovered from nesting *position* rather than from label vocabulary. The depth-map pass is where that claim is implemented, so I ablate it: an environment flag neutralizes Pass 1 and the detector falls back to the no-depth-map path it already uses when inference fails, meaning the off arm is the system's real graceful-degradation behavior rather than a strawman built for the paper.

Read Table 4 by level, not by its mean. Removing the depth map leaves domain recall untouched at 1.000 and indicator recall untouched at 1.000,

State	Role	Subset pp	Recall	Verified prec.	Code acc.	Desc. acc.	Depth map
AZ	dev	15	100.0	100.0	100.0 (4/4)	100.0 (4/4)	PASS
CA	dev	13	100.0	100.0	100.0 (25/25)	100.0 (14/14)	PASS
CO	dev	10	100.0	100.0	100.0 (7/7)	100.0 (4/4)	PASS
TX	dev	9	100.0	100.0	100.0 (8/8)	100.0 (3/3)	PASS
NV	validation	15	100.0	98.1	100.0 (46/46)	100.0 (3/3)	PASS
KY	validation	8	100.0	100.0	100.0 (44/44)	100.0 (26/26)	PASS

Table 2: Detector recall, verified precision, code and description accuracy, and depth-map pass/fail, per state; column definitions and the reason raw suite precision is omitted are in the text. All numbers are on the *_only_subset* corpus tier (§3), never full documents. Code accuracy is that of the recorded run and is not a stable point value where repeats disagree: NV 44–45/46 across 5 repeat runs against 46/46 recorded (§6.6). Source: `task1_20260826`, `task2_20260826`.

State	Role	Coverage	Field acc.	Fully correct	Standards	ID collisions
AZ	dev	100.0 (18/18)	99.7	17/18	45	0
CA	dev	100.0 (21/21)	100.0	21/21	94	0
CO	dev	100.0 (9/9)	100.0	9/9	48	0
TX	dev	100.0 (8/8)	100.0	8/8	25	0
NV	validation	100.0 (24/24)	100.0	24/24	24	0
KY	validation	100.0 (26/26)	100.0	26/26	26	0

Table 3: Parser coverage, per-field accuracy, fully-correct standards, standards emitted, and `standard_id` collisions, per state. Collisions are counted after the identifier resolver (§5.3), which renames any it finds, so that column is zero by construction; whether the resolver had to act is reported in §6.1. All numbers are on the *_only_subset* corpus tier (§3), never full documents. Source: `task1_20260826`, `task2_20260826`.

Level	Map on	Map off
domain	100.0	100.0
strand	100.0	92.0
sub_strand	100.0	83.3
indicator	100.0	100.0

Table 4: Depth-map ablation: detector recall (%) with the Pass-1 depth map on and off, pooled over all six states. Read by level, not by its mean; per-state behaviour, off-arm ranges and the regression cases are in the text. *_only_subset* corpus tier. Source: `task3_20260826`.

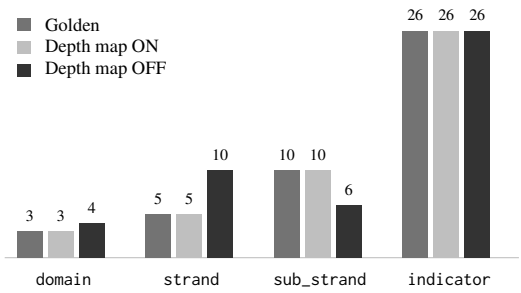


Figure 1: Elements emitted per hierarchy level on Kentucky, whose detector golden is the corpus's only *detection-exhaustive* one, with the depth-map pass enabled and ablated. The enabled arm reproduces the golden distribution exactly; ablated, strand inflates and sub-strand deflates while domain and indicator hold: level *collapse*. `_only_subset` corpus tier.

while strand falls to 0.920 and sub-strand to 0.833. That is the claim, measured: the levels that survive are the ones with a distinctive surface form (a document's top division is typographically obvious, and its leaves are the numbered or lettered items), and the levels that degrade are exactly those whose identity is positional, recoverable only from what they sit between.

Kentucky shows the mechanism (Figure 1). Its golden annotates 3 domains, 5 strands, 10 sub-strands and 26 indicators; the on arm reproduces this exactly, while the off arm emits 4, 10, 6 and 26. Three repeated runs per arm, recorded at an earlier detector version (before the Pass-1 sampler was replaced, so kept separate from Table 4), show the same shape every time: the on arm matches the golden, and the off arm inflates strand (5 → 8/7/10), deflates sub-strand (10 → 7/9/6) and leaves indicators at 26. This is level *collapse*, and the regression cases show its cause: without the map, elements printed as `Benchmark N.N` are promoted to strand because of their label rather than their position, exactly the behavior the method exists to avoid.

Two qualifications. First, **the effect is conditional, not uniform**: only Colorado and Kentucky lose recall, while Texas and Nevada keep recall 1.000 and lose *code* accuracy instead (Texas 8/8 → 5/8 on an identical set of detections). Arizona and California are insensitive on every metric, but that is an artifact of their goldens rather than a result: both are small spot checks against far larger

detection sets, so recall saturates either way, and they are the states least able to show the effect. Second, **off-arm recall is a range, never a point value**: across three runs per arm per state at the earlier code version, Colorado spans 0.71–0.86 and Kentucky 0.89–0.98, and the recorded run at the current version (0.86 and 0.89) falls inside both ranges. The direction never reverses in any sample, and the on arm is 1.000 with zero variance across all eighteen runs, but the magnitude on any single run is a draw. One regression case that failed at n=2 passed at n=3; I report it as unstable rather than as evidence.

6.3 Comparison: A Rule-Based Structure Extractor

To quantify what the language model contributes, I compare against a rule-based structure extractor built on the cues such documents actually provide: numbering patterns, capitalization, indentation and line shape. It is graded by the same suite against the same goldens, through the same code path, so the two columns are directly comparable. It was developed against the four development states with Nevada and Kentucky untouched, and it carries no per-document branch.

The per-level row is the result. The rule-based arm ties the language model exactly at domain (13 of 13 in both) and then collapses: strand 0.240, substrand 0.667, indicator 0.137, against 1.000 at every level. Reported as a mean this would read "1.000 versus 0.500" and would hide the only thing worth knowing. Typography identifies a document's top division in all six states and identifies nothing reliably below it, because below the top division the distinctions are positional.

The dominant failure mode is omission rather than misclassification: most elements the baseline misses are never emitted at all, rather than emitted at the wrong level (43 of 46 in Nevada, 31 of 44 in Kentucky). The rules do not so much mislabel the hierarchy as fail to detect it.

The validation-state collapse needs its repair attached. Nevada at 0.065 is the most dramatic number in the table and the least trustworthy as stated: it is dominated by a single token shape (`SS.ID.PK1`, a multi-letter code segment) that none of the four development documents prints. Widening the pattern after the fact recovers 0.522, and a comparable repair takes Kentucky from 0.295 to 0.409. I adopt neither, because both were motivated by inspecting the two documents after the recorded run. The claim that survives the repair is the one worth making: rules encode the shapes their author has seen, and even generously repaired they reach roughly half the language model's recall on documents they were not written against. The comparison on these two states is not symmetric: their failures informed general fixes to the pipeline (§5.7) and none to the baseline. The development states carry no such asymmetry, and the baseline falls short there too, at 57–80% recall against 100%.

Two cautions about the precision columns. Raw precision is not a quality ordering here: Arizona shows the rule-based arm *above* the language model (0.500 versus 0.417) purely because it emits fewer in-scope detections against a five-element spot-check golden. I also omit verified precision from this table deliberately. It asks whether a detected title appears in the source text, a test for *invention*, and an extractor that copies verbatim cannot invent, so the baseline scores 1.000 in five of six states against the language model's 0.9966. That number is real and substantively backwards. Finally, the baseline has no Pass 1, so its depth-map column reads ABLATED rather than failed, and it is deterministic, so one run suffices.

6.4 Comparison: A Single Whole-Document Prompt

The rule-based extractor of §6.3 answers what a regular-expression approach cannot do. It does not answer the other obvious question: why not hand the model the entire document in one prompt and read the JSON back? §5.1 argues against that; this measures it. The arm is one call per document over the whole extraction, with no Pass-1 depth map, no chunking and therefore no merge, and none of the deterministic repairs of §5.3. The prompt body, model, temperature, extractions and grader are identical to the full method's, so a difference is attributable to the architecture rather than to prompting.

It is not a strawman: on Arizona and Nevada it matches, at recall 1.000 with full code accuracy, and Nevada is one of the two validation states and prints a namespace that skips a level, so this is not an easy case. Where it loses, it loses in two distinct ways. On Kentucky and Texas recall holds while code accuracy falls, $44/44$ to $28/44$ and $8/8$ to $5/8$, and the failures are precisely the repairs' absence: the model sampling its own abbreviation rule, and codes left unanchored to their parents. Since a code composes the identifier, each

	Recall		Raw precision		Code acc.		
State	**LLM**	**Rule**	**LLM**	**Rule**	**LLM**	**Rule**	**Rule: found, wrong level**
AZ	100.0	80.0	41.7	50.0	4/4	3/3	0/5
CA	100.0	64.0	20.5	18.2	25/25	16/16	0/25
CO	100.0	57.1	16.7	8.2	7/7	4/4	1/7
TX	100.0	62.5	32.0	13.5	8/8	4/5	1/8
NV	100.0	6.5	88.5	16.7	46/46	1/3	0/46
KY	100.0	29.5	100.0	100.0	44/44	13/13	0/44
Pooled recall by level, all six states							
domain	100.0	100.0	LLM 13/13, rule-based 13/13				
strand	100.0	24.0	LLM 25/25, rule-based 6/25				
sub_strand	100.0	66.7	LLM 24/24, rule-based 16/24				
indicator	100.0	13.7	LLM 73/73, rule-based 10/73				

Table 5: Rule-based baseline versus the LLM detector, graded by the *same* suite against the *same* goldens as Table 2. **Raw precision is shown for both arms and is not a quality measure** except for KY, whose golden is detection-exhaustive; elsewhere it tracks annotation coverage and rewards under-emission (§6.3). *_only_subset* corpus tier, never full documents. Source: `task4_20260826`.

is a different primary key for the same standard. On Colorado recall itself falls to 0.429, with the level-collapse signature the depth-map ablation produces.

California is lost entirely, and that failure is architectural rather than statistical. Its single call emitted exactly 16,000 output tokens, the configured maximum, truncating the JSON mid-object so that nothing parsed and *none* of its 25 golden elements were recovered, against $25/25$ for the full method. One prompt has one output budget for an entire document, so exceeding it loses the document; chunking bounds the same overflow to a chunk. Arizona shows how narrow the margin is: it succeeded while emitting 14,292 tokens, within 11% of the same ceiling, so a success here does not predict the next document, and the margin is invisible in the score.

Scope bounds what this carries. Every subset extraction is between 3.7K and 7.9K tokens and fits in one prompt, so a good score here would mean the architecture is unnecessary *at this tier* and would say nothing about the tiers where a document does not fit and chunking is not optional. This arm is a single draw per state, which is why I do not tabulate it; the California result is reported with more confidence than the rest because its mechanism is identified rather than inferred, the output length equaling the configured cap exactly. Source: `task10_20260905`.

6.5 What the Confidence Score Measures

The detector emits a confidence score on every element, and nothing in the pipeline thresholds it (§5.5). The measured distribution is why. Across 379 elements the score takes just five distinct values, all in $[0.90, 0.97]$, and 94.2% fall at 0.95 or above; the band the prompt reserves for a guess (< 0.70) is never used. What variation exists tracks the document rather than correctness: 16 of the 22 middle-band elements are Kentucky's, yet against the corpus's only exhaustive golden Kentucky scores 1.000 on recall, code accuracy and verified precision while carrying the lowest mean confidence of any state. A review gate built on this score would mostly be a gate on noise. The full distribution, its run-to-run drift, and one observation about hallucination are in Appendix D.

6.6 Stability: What Changes Between Identical Runs

Every number reported so far is a single run of a sampling model, so the question a reader should ask is how much of it survives re-running. I ran the detector five times and the parser six times per state on identical frozen inputs with caching disabled, deliberately split across two days.

The instability is concentrated rather than diffuse. Four of six states reproduce perfectly on each suite: California, Colorado, Texas and Kentucky show zero detector disagreements across five runs, and Arizona, Colorado, Texas and Kentucky show zero parser disagreements across six. The movement that exists is confined to three places, and each is a mechanism I can name. Arizona's detector alternates on whether it emits three of its sub-strands (present in four runs, absent in one), which changes output size without touching any

	ids	unstable	rate
Detector			
AZ	67	3	0.0448
CA	51	0	0.0000
CO	61	0	0.0000
TX	33	0	0.0000
NV	48	2	0.0417
KY	44	0	0.0000
pooled	**304**	**5**	**0.0164**
Parser			
AZ	45	0	0.0000
CA	33	4	0.1212
CO	48	0	0.0000
TX	25	0	0.0000
NV	24	5	0.2083
KY	26	0	0.0000
pooled	**201**	**9**	**0.0448**

Table 6: Run-to-run stability, **`_only_subset` corpus tier**. Five detector runs and six parser observations per state, all `--no-cache`, split across two days. *ids* is the number of distinct element identities compared, *unstable* how many differed in any graded field. **These rates are lower bounds, not estimates** (§6.6). Source: `task5_20260830`.

graded element. California's parser intermittently leaves the structural label inside an indicator code, emitting `ELD.1.0.VOCA.Foundation 1.1.BROA` in two of five comparisons. Nevada moves on both suites: a domain code that drifts between the document's `S` and the domain's full name `Science`, and a strand attachment that changes which of two headings three standards hang from.

The California case is caught rather than stored. All twelve of those malformed codes contain whitespace, so the shape check at the validation boundary (§5.3) rejects every one before persistence. This is the clearest evidence available that the check is load-bearing rather than defensive decoration: the defect it was written for fires at a few percent per run, and when it fires the records do not reach the database.

Why the runs were split across days. Repeated runs inside one session understate variance, and this measurement demonstrates it directly. Nevada's detector emitted the same set of 54 elements in each of the three runs executed in one evening (one of them differing only in the Science domain's code), then dropped one indicator in *both* runs executed the following day. Five back-to-back runs would have reported that element as perfectly stable.

A single run is not a safe basis for a claim. An earlier recording of this system measured Nevada's detector code accuracy at $46/46$ and I initially recorded it as a gain. It does not reproduce: five fresh runs on identical code, the same extraction and the same golden give 44, 45, 44, 44, 44. Nevada's code accuracy is properly reported as a *range* of 44–45 of 46, and the earlier point value was the top of its distribution rather than its center.

The same caution applied to a single-run ablation arm of my own, and repeating it at $n{=}5$ per arm *retired* the claim rather than confirming it. I had attributed Nevada's two domain-code failures to the Pass-1 sampler on one draw per arm; across five draws, one of those codes fails in *every* draw of *both* arms, and at $n{=}1$ that arm had simply drawn the $46/46$ run above. What survives is a rate rather than a cause: the layout-stratified sampler gives mean code accuracy 0.965 against the stride sampler's 0.935, its worst draw beating the stride sampler's best. However, it also drops a golden element in two draws of five where the stride sampler drops none, so it is better on average, but not better in every respect. Neither the sampler's motivating evidence nor the depth-map ablation is affected: the first is a far larger failure elsewhere, a four-level document reported as three (§5.2), and the second is three runs per arm across six states whose direction never changes sign.

What this does and does not license. These rates are **lower bounds, not estimates**. A defect firing in a minority of runs survives five clean draws comfortably. The California defect above appears in two of five comparisons, and an earlier investigation of a different parser defect, the bare indicator code of §5.3, found it in one graded run of six within a single invocation and in ten of twenty cached runs across code versions. A zero in Table 6 means I did not observe instability at $n{=}5$, not that the component is deterministic. This is why the table reports denominators beside every rate, and why the pipeline's correctness argument rests on the validation boundary rather than on the model agreeing with itself.

6.7 Scale: the Batched Path on Multi-Batch Documents

Every quality measurement above is computed at the `_only_subset` tier, where each document produces at most five chunks and three domains. Both batching layers therefore collapse to a single batch, the Step Functions `Map` runs one iteration, and the merge is a no-op, so those tables cannot support a

	KY	CO
Pages processed	52	41
Batching		
Detection chunks	18	17
Detection batches	4	4
Parse batches	9	9
Elements, raw → merged	363 → 329	390 → 300
Input tokens		
Depth map	12,153	19,613
Detection	201,574	223,276
Parsing	164,783	147,315
Total	**378,510**	**390,204**
Output tokens		
Depth map	438	415
Detection	64,530	47,125
Parsing	85,205	83,862
Total	**150,173**	**131,402**
LLM calls	45	45
Wall clock	13m19s	9m52s
Cost (USD)		
Depth map	$0.0143	$0.0217
Detection	$2.6211	$2.2945
Parsing	$1.7724	$1.6999
Total	**$4.41**	**$4.02**

Table 7: Batched-path scale, **`_trimmed` corpus tier**: each document's standards content *in full*, and the only table here not at the `_only_subset` tier (§6.7). Each stage runs on the model assigned in §5.4. **Tokens are the primary measure**; cost is derived from them at published Bedrock on-demand rates, recomputed at table-build time from the pipeline's own pricing constants. The depth-map pass is 0.3–0.5% of a run's cost, detection 57–59%. Source: `task6_20260830`.

claim about batching. This subsection measures the batched path directly, on documents large enough to exercise it.

Table 7 is not the subset tier, and the distinction is easy to misread in both directions. These runs are the `_trimmed` tier: each document's standards content in full, with front matter, expository essays and appendices removed. Kentucky's 52 pages are all of Kentucky's standards, not 43% of them. What the tier reduces is page count, and therefore cost; it does not reduce coverage. Equally, these are not numbers for the published PDFs, and the token and latency figures should not be extrapolated to them.

At this tier the batching path is genuinely exercised: 18 and 17 detection chunks resolve into four `Map` iterations per run, parsing into nine batches, and the merge removes 34 and 90 duplicate elements respectively. A no-op merge cannot remove 90 elements, so the reconciliation the architecture performs is doing real work rather than passing data through.

The stage breakdown is where the model-assignment principle of §5.4 is tested. Detection is the only stage invoked once per chunk and accounts for 57–59% of each run's cost. The depth-map pass, the component the ablation above shows carries the intermediate levels, runs once per document on the cheapest model and costs between 0.3% and 0.5% of a run, about 1.4 cents on Kentucky and 2.2 on Colorado. The most load-bearing component in the method is very nearly the cheapest one, which is the outcome the assignment was designed for.

Tokens are the primary measure and cost is derived from them at published on-demand rates, recomputed at table-build time from the pipeline's own pricing constants rather than transcribed by hand. Neither run was throttled, no detection chunk returned empty, and the validator rejected no records, so these figures describe a clean pass rather than a degraded one.

6.8 Quality at Full-Document Scale

Every quality figure above is computed at the *_only_subset* tier, and the obvious objection is that subset-tier quality may not survive a whole document: a larger run chunks further, and chunk boundaries are where a detector can duplicate an element or lose one. §6.7 shows that the *_trimmed* tier genuinely exercises the batched path, but it measures cost, not quality. Here I grade that path's output.

Two reference sets. The first regrades the existing *_only_subset* goldens *inside* the trimmed-tier run, translating page numbers through the recorded page map of §3. It needs no new annotation and isolates the variable that matters: the same pages and the same golden, in a larger neighborhood. Kentucky and Colorado both come back with zero elements absent and zero ungrounded detections. Because this covers only the subset's pages, I also annotated new detector and parser goldens for Kentucky's entire trimmed tier (277 elements and 207 normalized standards, seeded from one recorded run and checked page by page against the published PDF) and graded three further runs against them (Table 8). Since these goldens were verified against the document rather than against the run being graded, a miss reflects the run, not the annotation.

Detection recovers the whole reference set at scale. All **277 of 277** verified elements are found by all three runs. Not one is missed, and not one

	1	2	3	sd
Detector, 277 verified elements				
Elements emitted	328	327	328	0.6
Absent from run	0	0	0	0.0
Hallucinations	0	0	0	0.0
Recall (level+title)	1.000	1.000	1.000	0.000
Recall (strict key)	0.592	0.588	0.592	0.002
Spurious age_band	164	164	164	0.0
Surplus duplicates	51	50	51	0.6
Parser, 207 verified standards				
Standards parsed	214	220	220	
Coverage	0.807	0.927	0.841	0.062
Field accuracy	0.997	0.994	0.994	0.002
Fabricated .N keys	10	11	11	0.6
Collisions†	0	0	0	0.0

Table 8: Quality at whole-document scale: Kentucky, three independent runs, **`_trimmed` corpus tier**: the document's standards content *in full*, and not the `_only_subset` tier the headline quality tables use (§6.8). Recall is measured against goldens checked against the document for omissions as well as for correctness (§7.3). **Coverage is a range and must not be read as its mean.** † zero by construction: the resolver renames every collision before the check runs, so the informative count is the fabricated-key row above. Source: `task13_20260907`.

ungrounded element is emitted, across a $6.5\times$ increase in page count and four detection batches per run.

One field accounts for every anomaly. Kentucky prints a `THREE AND FOUR YEAR OLDS` banner at the head of each page. At the subset tier the detector ignores it and every element comes back with a null age band, matching the golden. At the trimmed tier it does not: 164 elements carry the banner as their own `age_band` in each of the three runs, with a standard deviation of zero on the count, and 159 of those elements are the same in all three runs (two runs agree exactly; the third differs on five). That makes this a **systematic misreading rather than sampling noise**. Three consequences follow, and together they are the entire difference between the strict figures in Table 8 and the ones above:

1. elements fail the evaluation's matching key on that field alone, which is why strict recall reads 0.59 while recall by level and title is 1.000;

2. `_dedup_elements` keys on `age_band`, so the two spellings of one element never merge, and roughly 15% of detector output survives as surplus duplicates;

3. those duplicates collide in parsing and are separated by the identifier resolver's numeric fallback, so ten to eleven fabricated primary keys per run reach persistence, naming standards that do not exist.

Colorado is the control. Its document has no age-band column and no such banner, and in the same measurement it produces **zero** duplicates and zero fabricated keys over 240 standards. That contrast is what makes the attribution a measurement rather than a hypothesis.

Parser coverage is a range; parser field accuracy is not. Coverage spans twelve points across the three runs while field accuracy holds at 0.995 ± 0.002. The swing tracks the dropped-standard count exactly, and the dropped standards are those the run assigned a narrower age band than the document carries. **A single coverage figure for this document would be a draw from that range**, and I report all three rather than their mean.

A figure that looks reassuring and is not. Identifier collisions are zero by construction: the resolver renames every collision before any check runs, so the informative count is the fabricated keys above.

What this establishes. Detection quality survives full-document scale on the one document where it can be checked: complete recovery of every element in the document, no hallucination, in three independent runs. The defects that scale introduces are concentrated in one field, are systematic rather than random, and are documented rather than repaired, because the repair changes a prompt and would invalidate every frozen measurement in this paper (§7). Source: `task9_20260905` and `task13_20260907`.

7 Discussion and Limitations

When deploying a system like this, the system's limits matter as much as its results. This section separates what the measurements support from what they do not, and flags which limits are unmeasured rather than measured.

7.1 What the results support

The central claim survives the strongest test this corpus allows. Classifying by nesting position rather than by label vocabulary yields recall of 1.000 at every hierarchy level in all six states, including the

two validation states, with no code path for any of them. That is a validation result rather than a held-out one: Nevada and Kentucky were used to check, and in places prompted, general fixes during development (§5.7), and no state in the corpus was untouched by development. The two comparisons locate that result rather than merely asserting it, and they locate it at the same boundary from opposite sides: ablating the depth map degrades exactly the levels whose identity is positional and leaves the others untouched, while the rule-based extractor matches the language model at a document's top division and collapses beneath it. Typography identifies that top division reliably and identifies almost nothing below it.

7.2 Corpus scope, and what the tiers do and do not mean

Every headline quality number in this paper is computed at the `_only_subset` tier: roughly one to two domains per state, 70 pages in total, hand-trimmed and exhaustively annotated. Trimming by hand favors the system, since it removes the front matter, essays and appendices a whole document would add. The tier makes annotation affordable, but it is a genuine reduction in coverage: a subset metric is not a whole-document metric, and no table at that tier should be read as one. The one exception, §6.8, grades whole-document runs against a golden of its own and is kept in its own table.

The scale measurements are a different tier and a different kind of claim. They run at `_trimmed`, which removes only non-standards matter (front matter, guiding-principles essays, appendices, acknowledgements) and **retains every standard in the document**. Kentucky's 52 processed pages are all of Kentucky's standards, not 43% of them, and Colorado's 41 are the complete Ages 3–5 publication. A trimmed-to-published page ratio is therefore never a coverage fraction, and I am explicit about this because the opposite reading is the more natural one and would understate the result.

What remains limited about scale is **page count, as a cost and batching question rather than a coverage one**. These runs chunk 18 and 17 times across four `Map` iterations; their token and latency figures should not be extrapolated to untrimmed page counts, and no document as large as Arizona's 217-page publication has been run end to end. Trimming also remains preprocessing that a deployment would have to perform or absorb. Colorado carries one further qualification. Its out-of-scope content is other *age bands*, the birth-to-three and five-to- eight portions of a 187-page publication, not standards removed by trimming. Every Colorado figure in this paper is the Ages 3–5 band.

7.3 What the goldens can and cannot establish

The detector goldens vary a great deal in density, and that governs what each can establish. In the four development states they are deliberate spot checks: 5, 25, 7 and 8 annotated elements against detection sets of 67, 122, 61 and 33. Because the evaluation suite counts every unmatched in-scope detection as a false positive, *raw* precision on those states largely measures annotation coverage rather than detector quality, and I do not report it as quality. Precision instead comes from a manual audit of all 297 in-scope detections across the six subset-tier documents, which found a single hallucinated element.

The two validation states are annotated densely rather than as spot checks, and they separate two properties that are easily conflated. Nevada's golden is content-exhaustive (46 elements covering every distinct structural element the document prints) but not *detection*-exhaustive, because the document reprints several headings on a second page spread and the detector correctly finds them twice; at 46 golden against 52 in-scope detections its raw precision has a ceiling below 1 no matter how well it performs. Kentucky's golden is the one that is detection-exhaustive, 44 against 44, so its precision is a genuine hallucination rate rather than a coverage figure. It is also, for that reason, what made quality-at-scale evidence available without new annotation: those 44 elements are graded inside a trimmed-tier run in §6.8.

The trimmed-tier goldens support both precision and recall. The two Kentucky goldens used at that scale (277 elements and 207 standards) were seeded from a recorded run and then checked against the published PDF in both directions. Every seeded row was confirmed against its page, which supports precision. Seeding alone cannot support recall, since an element no run emits would be missing from the golden too, so the document was also checked for elements no run had emitted; there were none. Recall at that scale is therefore measured against the document, not only against what the system produced.

Independent evidence corroborates that check at the middle levels. Every distinct `Benchmark N.N`

identifier the document prints (18 of 18) and every `<Domain> Standard N` heading (15 of 15) appears in the golden, recovered from the document text rather than from any run.

Colorado contributes recall evidence but no precision figure. Its golden remains a seven-element spot check and is deliberately not being extended, so at the trimmed tier it contributes recall and duplication evidence (zero elements absent, zero duplicates, across four detection batches on the corpus's hardest layout) and no precision figure at all. Its raw precision there is an annotation-coverage artifact and is labeled as one in the recorded output. The corpus therefore has exactly one state whose precision is a quality figure, at either tier, and that is a real limit on how far the evidence generalizes.

7.4 Nondeterminism, and why I do not claim determinism

The pipeline is not deterministic, and the stability measurements should be read as **lower bounds rather than estimates**. A defect firing in a minority of runs survives five clean draws comfortably, so a zero in my stability table means I did not observe instability at $n{=}5$, not that a component is stable.

Two of my own recorded results are the cautionary cases, and §6.6 reports both: a Nevada code accuracy of $46/46$ that five fresh runs give as 44, 45, 44, 44, 44, and a single-draw attribution of Nevada's domain-code behavior to the Pass-1 sampler that $n{=}5$ per arm retired rather than confirmed. Both were the top of a distribution read as its center. Where an experiment is repeated (the depth-map ablation, at three runs per arm across six states) I report the range and note that the direction never reverses, rather than quoting a single number.

The practical consequence is that correctness here rests on the validation boundary rather than on the model agreeing with itself. The clearest instance is California's intermittent structural-label defect: when it fires it produces twelve malformed identifiers, all of which contain whitespace and are rejected on shape before persistence.

7.5 A known defect, measured and deliberately not repaired

§6.8 reports that one field accounts for every anomaly the trimmed tier introduces: the detector attributes a page banner to individual elements, which defeats the deduplication key, which produces duplicate standards, which the identifier resolver separates with fabricated suffixes. I know the cause, I know the fix, and I have not applied it.

The fix is a prompt rule stating what the goldens already encode across all six states: that an element's age band comes from a side-by-side column that distinguishes it from its siblings, never from a banner or running header that applies to a whole page. That is a general document-structure principle rather than a per-state rule, so it belongs in the prompt by the discipline of §5.7. Applying it changes a prompt, which changes the hash that every recorded measurement in this paper is keyed to, and would require re-recording the headline, ablation and comparison arms.

I report the defect rather than repair it, and state the reason plainly: the cost of the repair is the invalidation of the evidence. That is the same cost-gating that governs every other change to the detector and parser, applied to a finding of my own, and a reader is entitled to weigh it. Two things make the deferral defensible rather than convenient. The defect does not affect any subset-tier number, where the banner is not attributed at all. And it is systematic rather than random: the count is 164 in each of three consecutive runs, and 159 of those elements are common to all three, so its effect is bounded and reproducible rather than a source of unquantified noise.

7.6 Limitations of the measuring instruments

Three properties of the evaluation tools themselves affect how their numbers should be read.

The false-positive audit tests for invention. Its verdicts turn on whether a detected title appears in the source text, which is the right question for a generative system and the wrong one for a copier: a rule-based extractor cannot invent and therefore scores near 1.000 by construction. Applied to the baseline the metric also detects something other than hallucination: California's five flagged rows are column *fusion*, where a table header has been merged into an adjacent title, rather than fabrication. I report raw precision for both arms instead, and treat the audit as informative only about the language-model arm.

The two evaluation suites do not grade the same detection. The detector suite re-runs detection in process against a frozen extraction (the direct path), while the parser suite runs the parser directly over detection output that the deployed batched path stored. On the recording this paper

uses, Arizona's direct-path detection holds 67 elements where the batched output the parser suite reads holds 65; earlier recordings have differed by more. The two suites also use independently annotated goldens with different shapes, a flat element list and nested normalized records. A detector number and a parser number from the same results directory therefore describe related but not identical objects, and should not be composed into a single end-to-end figure.

Extraction quality bounds everything downstream. The pipeline reads OCR and PDF text-layer output; a heading the extractor fuses or drops is not recoverable by any amount of downstream reasoning. Nevada's one confirmed hallucination originates this way, in a multi-column table that flattens into interleaved reading order.

7.7 Scope

The corpus is six US states of more than fifty jurisdictions, all in English, all early learning standards, and none of the six is a clean held-out test (§5.7). I make no claim about other document families, other languages, or other countries' standards traditions, though the method assumes only that a document has a nesting structure and prints some evidence of it. Neither code nor corpus is released (§10): the source documents are third-party state-agency publications that are not mine to redistribute, which limits direct reproducibility to the prompts and schema disclosed in the appendices.

8 Conclusion and Future Work

US early learning standards are public policy that software cannot read. I have shown that the obstacle is not text recovery but hierarchy recovery, and that an LLM can recover the hierarchy by classifying each element's nesting position against a depth map inferred per document, rather than by recognizing label vocabulary that no two states share. Across four development states and two validation states, detector recall at the annotated subset tier is 1.000 at every level, a manual audit of all 297 in-scope detections there finds one hallucinated element, and parsing resolves those elements into a single canonical schema without the identifier resolver ever having to separate two standards. Ablating the depth map degrades exactly the levels whose identity is positional and leaves the others untouched; a rule-based extractor matches the method at a document's top division and collapses beneath it. The system carries no per-state code path, which is what allows a new state to be added without new code; the prompts' worked examples are the one place the corpus's own documents appear, and they are itemized rather than left implicit.

Quality at full-document scale is reported for one document. On Kentucky's trimmed tier, all 277 elements of the document's verified golden are recovered in each of three independent runs, with no hallucination, and parser field accuracy is 0.995. Scale also brings a defect the subset tier never shows: a page banner read as an age band leaves parser coverage between 0.81 and 0.93 across the three runs and produces ten or eleven fabricated primary keys per run (§7.5). What remains is breadth. The corpus is six states of more than fifty. The first state examined only after development is frozen would be the paper's first true held-out test, and enough states would constitute the national machine-readable dataset that does not currently exist.

Two further directions follow from the artifact rather than the method. Exposing the normalized store through a tool-callable interface would let external applications query standards directly, which is the form curriculum and planning tools need them in. And the per-element verification state invites a targeted-review question I have deliberately not answered: given that confidence is uninformative (§6.5), what *would* identify the records most worth an expert's attention? My stability measurements suggest one candidate. Disagreement across repeated runs is cheap to compute and, unlike confidence, is concentrated in exactly the places where my own defects live.

9 Ethics and Broader Impact

No personal data. This work processes published policy documents issued by state education agencies. It involves no student, teacher or family data, no personally identifying information of any kind, and no human subjects. The only people named anywhere in the pipeline are the authenticated editors who verify extracted records, whose identity is bound from their session rather than supplied by a model.

Stewardship of what the system produces. Early learning standards shape curriculum, assessment and teacher preparation, so a misextracted standard is not a neutral error: it could propagate

into materials that reach children. This is the reason the system emits a confidence score but gates nothing on it (§5.5), and the reason every record carries the page of the source PDF it came from: the design assumes a human will check, and makes checking cheap, rather than assuming the extraction is right. My own measurements support that stance. The pipeline is not deterministic, its stability figures are lower bounds rather than guarantees, and I found a recorded result of my own that did not reproduce. I would regard deploying these outputs into anything carrying regulatory or assessment weight without expert verification as a misuse of them.

Not a substitute for the source. The normalized corpus is a navigation aid over documents that remain authoritative. Nothing here reinterprets a standard, changes its wording, or merges jurisdictions into a common framework; the canonical schema records how each state organizes itself, including where a state uses fewer levels than the schema allows. Treating a normalized record as the standard, rather than as a pointer into the document that issues it, would be a misreading of what the system does.

Intended beneficiaries and misleading comparisons. Making fragmented standards queryable mainly helps those without the resources to do it by hand: small districts, individual educators, and developers building for them, as against large publishers who can afford manual normalization. That is the impact I intend. The corresponding risk is that a machine-readable corpus makes cross-state comparison easy enough to invite comparisons the documents do not support. States differ in what they choose to make explicit, and a state with fewer recorded indicators cannot be said to have a lower bar for quality.

10 Artifacts Statement

Neither the implementation nor the evaluation corpus is released. Disclosure is in-paper: the appendices give the detector and parser prompts (Appendix A, with their complete unabridged text shipped alongside this paper as the ancillary file `anc/prompts.txt`), the canonical schema (Appendix B), representative annotated examples (Appendix C), and the corpus table naming every source document with its publisher, year and retained page ranges (Appendix E). Those are the parts a reader needs to evaluate or reimplement the method; the rest is engineering around them.

The source documents are third-party state-agency publications. Unlike US federal works, which are excluded from copyright, state government works have no uniform public-domain status, so I do not redistribute them; the corpus appendix cites each document's origin so it can be obtained from its issuing agency instead. The hand-annotated golden sets are derived from those documents and are withheld for the same reason.

I state the consequence plainly: this limits direct reproducibility. A reader can reimplement the method from the prompts and schema and apply it to documents they obtain themselves, but cannot re-run my exact measurements. To make the reported numbers as checkable as the constraint allows, every tabulated result in this paper is generated from a recorded JSON artifact rather than transcribed by hand; the few figures in the prose that come from engineering notes instead are marked as such where they appear. Each recording carries the pipeline source version and model identifiers that produced it, and each table is regenerated from those artifacts at build time. Every table caption names the artifact behind it by task and recording date (`task8_20260904`, say), and those recordings are retained and available on request; the date is load-bearing, because several measurements were re-recorded when the code version moved and the tag is what distinguishes one freeze from another. Where a figure could not be verified against an authoritative source I say so rather than reporting it.

This paper is distributed under the arXiv nonexclusive license to distribute. Copyright is retained by the author; no redistribution or derivative use of the text is granted.

References

1EdTech Consortium. 2017. Competencies and academic standards exchange (CASE) specification. https://www.1edtech.org/standards/case. Version 1.0 (2017); version 1.1 released 2025. Accessed August 2026.

Vikram Aggarwal, Jay Kulkarni, Aditi Mascarenhas, Aakriti Narang, Siddarth Raman, Ajay Shah, and Susan Thomas. 2025. Information extraction from fiscal documents using LLMs. ArXiv:2511.10659. Presented at the AI for Financial Inclusion, Risk Modeling and Resilience in Emerging Markets workshop at ACM ICAIF 2025.

Monica Agrawal, Stefan Hegselmann, Hunter Lang,

Yoon Kim, and David Sontag. 2022. Large language models are few-shot clinical information extractors. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1998–2022, Abu Dhabi, United Arab Emirates.

Lukas Blecher, Guillem Cucurull, Thomas Scialom, and Robert Stojnic. 2023. Nougat: Neural optical understanding for academic documents. ArXiv:2308.13418.

Holly Devaul, Anne R. Diekema, and Jonathan Ostwald. 2011. Computer-assisted assignment of educational standards using natural language processing. *Journal of the American Society for Information Science and Technology*, 62(2):395–405.

Education Commission of the States. 2024. 50-state comparison: Early care and education governance. https://www.ecs.org/50-state-comparison-early-care-and-education-governance-2024/. Accessed August 2026.

Fabrizio Gilardi, Meysam Alizadeh, and Maël Kubli. 2023. ChatGPT outperforms crowd workers for text-annotation tasks. *Proceedings of the National Academy of Sciences*, 120(30):e2305016120.

Yupan Huang, Tengchao Lv, Lei Cui, Yutong Lu, and Furu Wei. 2022. LayoutLMv3: Pre-training for document AI with unified text and image masking. In *Proceedings of the 30th ACM International Conference on Multimedia*, pages 4083–4091.

Hyeon Seong Jeong, Sangwoo Jo, Byeong Hyun Yoon, Yoonseok Heo, Haedong Jeong, and Taehoon Kim. 2025. Zero-shot document understanding using pseudo table of contents-guided retrieval-augmented generation. ArXiv:2507.23217.

Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. 2023. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):248:1–248:38.

Geewook Kim, Teakgyu Hong, Moonbin Yim, JeongYeon Nam, Jinyoung Park, Jinyeong Yim, Wonseok Hwang, Sangdoo Yun, Dongyoon Han, and Seunghyun Park. 2022. OCR-free document understanding transformer. In *European Conference on Computer Vision (ECCV)*, pages 498–517.

Minzhi Li, Taiwei Shi, Caleb Ziems, Min-Yen Kan, Nancy F. Chen, Zhengyuan Liu, and Diyi Yang. 2023. CoAnnotating: Uncertainty-guided work allocation between human and large language models for data annotation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1487–1505, Singapore.

Haokun Liu, Derek Tam, Mohammed Muqeeth, Jay Mohta, Tenghao Huang, Mohit Bansal, and Colin Raffel. 2022. Few-shot parameter-efficient fine-tuning is better and cheaper than in-context learning. In *Advances in Neural Information Processing Systems*, volume 35, pages 1950–1965.

Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173.

Patrice Lopez. 2009. GROBID: Combining automatic bibliographic data recognition and term extraction for scholarship publications. In *Research and Advanced Technology for Digital Libraries (ECDL 2009)*, pages 473–474.

Jiefeng Ma, Jun Du, Pengfei Hu, Zhenrong Zhang, Jianshu Zhang, Huihui Zhu, and Cong Liu. 2023. HRDoc: Dataset and baseline method toward hierarchical reconstruction of document structures. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pages 1870–1877.

Robert Monarch. 2021. *Human-in-the-Loop Machine Learning: Active Learning and Annotation for Human-Centered AI*. Manning, Shelter Island, NY.

Eduardo Mosqueira-Rey, Elena Hernández-Pereira, David Alonso-Ríos, José Bobes-Bascarán, and Ángel Fernández-Leal. 2023. Human-in-the-loop machine learning: A state of the art. *Artificial Intelligence Review*, 56:3005–3054.

National Center on Early Childhood Quality Assurance. n.d. Early learning standards and guidelines. https://childcareta.acf.hhs.gov/sites/default/files/state_elgs_web_final_2.pdf. Office of Child Care, Administration for Children and Families, U.S. Department of Health and Human Services. Accessed August 2026.

Office of Head Start. 2015. Head start early learning outcomes framework: Ages birth to five. https://headstart.gov/school-readiness/article/head-start-early-learning-outcomes-framework. U.S. Department of Health and Human Services. Accessed August 2026.

Morgan S. Polikoff, Andrew C. Porter, and John Smithson. 2011. How well aligned are state assessments of student achievement with state content standards? *American Educational Research Journal*, 48(4):965–995.

Andrew C. Porter. 2002. Measuring the content of instruction: Uses in research and practice. *Educational Researcher*, 31(7):3–14.

Johannes Rausch, Octavio Martinez, Fabian Bissig, Ce Zhang, and Stefan Feuerriegel. 2021. DocParser: Hierarchical document structure parsing from renderings. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pages 4328–4338.

Oscar Sainz, Iker García-Ferrero, Rodrigo Agerri, Oier Lopez de Lacalle, German Rigau, and Eneko Agirre. 2024. GoLLIE: Annotation guidelines improve zero-shot information-extraction. In *The Twelfth International Conference on Learning Representations (ICLR)*.

Burr Settles. 2009. Active learning literature survey. Technical Report Computer Sciences Technical Report 1648, University of Wisconsin–Madison.

Stuart A. Sutton and Diny Golder. 2008. Achievement standards network (ASN): An application profile for mapping K-12 educational resources to achievement standards. In *Proceedings of the International Conference on Dublin Core and Metadata Applications (DC-2008)*, Berlin, Germany.

Zhen Tan, Dawei Li, Song Wang, Alimohammad Beigi, Bohan Jiang, Amrita Bhattacharjee, Mansooreh Karami, Jundong Li, Lu Cheng, and Huan Liu. 2024. Large language models for data annotation and synthesis: A survey. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 930–957.

Somin Wadhwa, Silvio Amir, and Byron C. Wallace. 2023. Revisiting relation extraction in the era of large language models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15566–15589, Toronto, Canada.

Jiawei Wang, Kai Hu, and Qiang Huo. 2025. UniHDSA: A unified relation prediction approach for hierarchical document structure analysis. ArXiv:2503.15893. Accepted at Pattern Recognition.

Jiawei Wang, Kai Hu, Zhuoyao Zhong, Lei Sun, and Qiang Huo. 2024. Detect-order-construct: A tree construction based approach for hierarchical document structure analysis. *Pattern Recognition*, 156:110836.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023. Self-consistency improves chain of thought reasoning in language models. In *The Eleventh International Conference on Learning Representations (ICLR)*.

Sabine Wehnert, Harikrishnan Changaramkulath, and Ivan Habernal. 2025. HiPS: Hierarchical PDF segmentation of doctrinal legal books. ArXiv:2509.00909. Accepted as a demo paper at ICAIL 2026.

Brandon T. Willard and Rémi Louf. 2023. Efficient guided generation for large language models. ArXiv:2307.09702.

Hangdi Xing, Changxu Cheng, Feiyu Gao, Zirui Shao, Zhi Yu, Jiajun Bu, Qi Zheng, and Cong Yao. 2024. DocHieNet: A large and diverse dataset for document hierarchy parsing. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1129–1142, Miami, Florida, USA.

Derong Xu, Wei Chen, Wenjun Peng, Chao Zhang, Tong Xu, Xiangyu Zhao, Xian Wu, Yefeng Zheng, Yang Wang, and Enhong Chen. 2024. Large language models for generative information extraction: A survey. *Frontiers of Computer Science*, 18(6).

Qingshu Xu, Hong Jiao, Tianyi Zhou, Ming Li, Nan Zhang, Sydney Peters, and Yanbin Fu. 2025. Automated alignment of math items to content standards in large-scale assessments using language models. ArXiv:2510.05129.

Yiheng Xu, Minghao Li, Lei Cui, Shaohan Huang, Furu Wei, and Ming Zhou. 2020. LayoutLM: Pre-training of text and layout for document image understanding. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 1192–1200.

Xu Zhong, Jianbin Tang, and Antonio Jimeno Yepes. 2019. PubLayNet: Largest dataset ever for document layout analysis. In *2019 International Conference on Document Analysis and Recognition (ICDAR)*, pages 1015–1022.

A Prompts

This appendix and the schema of Appendix B are what this paper discloses in place of a code release (§10). Pass 1 is short and is given whole. The detection and parsing prompts are long (together roughly 43,000 characters), so rather than print them end to end I reproduce the individual rules the body of the paper argues about, each verbatim and each labeled so it can be cited from §5. **The complete text of all three prompts, unabridged, ships with this paper as the ancillary file `anc/prompts.txt`**; nothing is withheld by the selection here.

All three are reproduced as the system emits them, with only the per-run document content replaced by a placeholder.

A.1 Pass 1: depth-map inference

Run once per document, on a layout-stratified sample of text blocks (§5.2). Its output is supplied to every subsequent detection call as the authority for what each nesting level is.

```
You are analyzing the structural skeleton of an early learning
↪  standards document. You will NOT extract individual
↪ elements — only the document's nesting hierarchy.

Read the sample below and identify how many distinct nesting
↪ depths the document uses, what each depth looks like (its
↪ prefix/label pattern), and one concrete example. The
↪ DEEPEST depth is always the leaf (individual learning goals
↪ /foundations/benchmarks).

This document's depths must then be mapped to a canonical 4-
↪ level hierarchy:
- depth 1 → "domain"
- depth 2 → "strand"
- depth 3 → "sub_strand"
- deepest depth → "indicator"

Rules:
- Fill the canonical levels FROM THE TOP DOWN. The first depth
↪  is always "domain", the deepest depth is always "indicator
↪ ", and any depth in between takes the next canonical level
↪ in order: the depth directly under domain is "strand", the
↪ one below that is "sub_strand". A document with fewer
↪ depths therefore drops the levels at the BOTTOM of that
↪ middle range — never "strand".
```

```
- A 4-level document (domain > group > sub-group > leaf) maps
↪ to: domain, strand, sub_strand, indicator.
- A 3-level document (domain > group > leaf) maps to: domain,
↪ strand, indicator. There is NO sub_strand — the single
↪ middle level is a STRAND, never a sub_strand.
- A 2-level document (domain > leaf) maps to: domain,
↪ indicator. There is no strand and no sub_strand.
- Use depth POSITION, not the document's labels. If a document
↪  uses "Sub-Strand" as the label for the second level (
↪ directly under domain), it is still a STRAND in our
↪ canonical hierarchy — and likewise a document that calls
↪ its single middle level "Section", "Concept", "Goal", or
↪ anything else still maps that level to "strand".

Every line is prefixed with its page and the horizontal
↪ position it was laid out at: `[Page N | x=0.09]`, where x
↪ is the line's left edge as a fraction of the page width.
↪ Use x to work out whether a page uses a side-by-side layout
↪ : lines of one column share nearly the same x, and a jump
↪ to a clearly different x means a different column. Lines
↪ you judge to be in the same column read as one continuous
↪ cell, even when lines from other columns are interleaved
↪ between them. Treat parallel columns as repetitions of ONE
↪ depth, not as separate depths.

Be deterministic and conservative. Do not speculate. If you
↪ cannot tell whether two depths are distinct, assume they
↪ are the same depth.

Output ONLY a JSON object with this exact shape:
{
  "doc_depths": [
    {"depth": 1, "canonical_level": "domain",     "
↪ label_in_doc": "<what the doc calls this>", "prefix_pattern
↪ ": "<regex-ish pattern e.g. 'ALL-CAPS HEADING' or 'N. <
↪ Title>: <desc>'>", "example": "<exact text from doc>"},
    {"depth": 2, "canonical_level": "strand|indicator", ...}
  ],
  "notes": "<one short sentence about the LEVEL LAYOUT only, e.
↪ g. 'age-band columns repeat one depth'. A later stage reads
↪  this as an instruction, so: never describe text below the
↪ deepest depth as examples/illustrations/anecdotes, and
↪ never say to ignore or skip anything. Empty string if
↪ nothing unusual.>"
}

DOCUMENT SAMPLE:

<<serialized document lines, one per row, each prefixed
  [Page N | x=<left edge as fraction of page width>]>>
```

A.2 Pass 2: per-chunk detection

The prompt opens with the canonical hierarchy, the layout-coordinate convention and the classification rule, then states 9 numbered extraction rules, then the output schema. The excerpts below are the classification rule and the rules cited in §5; rules 2, 2a, 6, 7 and 8 are given in full in `anc/prompts.txt`.

A.2.1 Classification is positional

The instruction the rest of the prompt is built around: an element's level comes from where it sits in the depth map, never from the word the document uses to label it. This is what makes the prompt transfer across states that share no vocabulary.

```
CLASSIFICATION RULE:
- If a depth map is provided, look up each element's depth in
↪ the document and use the `canonical_level` from that depth.
↪  Do NOT reclassify based on prefix style.
- If no depth map is provided, classify by nesting POSITION,
↪ never by what the document calls a level.
```

A.2.2 An element requires a heading that is present

Grounding, and the reason detection can afford to be chunk-local: a chunk that opens part-way through a section emits the children alone rather than reconstructing the absent parent from the position identifiers inside a child's code. The parent is recovered later, in parsing (§5.3).

```
1. Emit every structural element you see, even if its children
↪  are not in this chunk. Emit ONLY what you see: every
↪ element must be backed by a heading line that literally
↪ appears in this chunk, and its `title` must be text copied
↪ from that line. A position identifier you read inside a
↪ CHILD's code is NOT evidence that the parent's heading is
↪ present. A child code names its ancestors — an indicator
↪ coded `A.B.2.c` says it sits under the `2` at the level
↪ above — but that tells you only where the child belongs; it
↪  does not put the parent's heading on the page. When a
↪ chunk opens part-way through a section and an ancestor's
↪ heading was left behind in the previous chunk, emit the
↪ children alone and leave the ancestor out. A later chunk
↪ that contains the heading will supply it, and the parent is
↪  reconstructed downstream from the children's codes. Never
↪ reconstruct it here by reading the code apart.
```

A.2.3 Side-by-side age-band columns

Age-banded documents lay one skill across several columns, and the columns are peer indicators rather than one indicator with several descriptions. The rule is stated as a count – N columns must yield N elements – because the failure it guards against is a visually sparse column being silently collapsed into its neighbor.

```
3. Side-by-side age-band columns: emit ONE element PER column.
↪  Different age bands = different indicators, even when they
↪  share a code stem and title. Set `age_band` to the column
↪ label (e.g. "Early (3 to 4 ½ Years)", "PK3", "By 36 months
↪ "). Strip the age-band label from `title`. Put only that
↪ column's prose in `description`. If a row shows N age
↪ columns it MUST yield exactly N indicators — emit EVERY
↪ column even when a column's prose is short, nearly
↪ identical to its neighbor, or visually sparse. Never
↪ collapse or skip a column.
   - Spell each age-band label identically every time, using
↪ the document's exact glyphs (write "½", not "1/2").
```

A.2.4 Code resolution and the derived abbreviation

This is the longest rule in the prompt, and §5.2 relies on it more than on any other rule. It orders three places a printed code may live – inline, in a caption, or as the shared leading prefix of its descendants' codes – and only then falls back to deriving one from the title. The abbreviation procedure at the end is the rule that is additionally executed in Python, for the determinism reasons given in §5.2.

```
4. `code`: REQUIRED on every element you emit. It is NEVER `
↪ null`, never an empty string, and the key is never omitted
↪ — an element without a code cannot be stored, and emitting
↪ one costs us the whole chunk it sits in. If the document
↪ prints no code for the element, the abbreviation procedure
↪ at the end of this rule ALWAYS supplies one; that is
↪ precisely what it is for, so "uncoded" is never a reason to
↪  answer `null`. The `ABSENCE IS null` instruction in rule
↪ 8 governs `description` and nothing else — it does NOT
↪ apply to `code`. Now: use the document's code if present.
```

↪ Before you decide none is present, LOOK for it — a document
↪ often prints an element's code somewhere other than on its
↪ heading line, and a code you can RECOVER from the page
↪ ALWAYS beats one you would derive from the title. Search
↪ these three places, in this order, and stop at the first
↪ that yields a code:
   (i) ON THE HEADING LINE — a code printed inline with the
↪ title ("1.0", "I.A.2", "PK3.I.A.2"), including the `<Label>
↪ <id>` form described in the bullets below. This always
↪ wins: an element that prints its own id keeps it, even when
↪ its descendants' codes are built on a different stem.
   (ii) IN A CAPTION BESIDE THE HEADING — a code the document
↪ prints just above, below, or beside the title rather than
↪ on it: inside parentheses, on a small caption line, in a
↪ table or column header, or in a lead-in that names the
↪ group of items which follows. A heading "Working Memory"
↪ whose next line reads "Indicators (CD.WM)" has code "CD.WM
↪ ". Take only the IDENTIFIER — the caption's surrounding
↪ words ("Indicators", "Standards", "Codes") name what the
↪ caption points at and are not part of the code — and copy
↪ it VERBATIM, dots included.
   (iii) FROM ITS DESCENDANTS' CODES — the heading prints no
↪ code of its own anywhere, but the elements BENEATH it carry
↪ dotted codes that all begin with the same segments. An
↪ ancestor's code is the COMMON PREFIX of its descendants'
↪ document codes. Indicators coded "CD.WM.PK1", "CD.WM.PK2",
↪ "CD.AT.PK1", "CD.AT.PK2" say that the group heading above
↪ the first two is "CD.WM", the group above the last two is "
↪ CD.AT", and the level above all four is "CD". Emit the
↪ WHOLE shared prefix with its dots ("CD.WM") — never just
↪ its last segment ("WM"), and never an abbreviation of the
↪ title.
      Peel exactly ONE segment per level as you move UP, so a
↪ recovered prefix never reaches past the level directly
↪ below it: the parent of an indicator coded "CD.WM.PK1" is "
↪ CD.WM", and the level above THAT is "CD". A level that
↪ prints its own code under (i) takes that code and consumes
↪ NO segment — keep peeling from where you were for the level
↪ above it. Peel this way even when the chunk happens to
↪ show only one group under a heading, where the raw shared
↪ prefix would over-reach ("CD.WM" for a level that is really
↪ "CD").
      Take a prefix only when the evidence is real:
      - The descendant's code must have MORE THAN ONE dot-
↪ separated segment. A single-segment code ("1", "a", "VOCA")
↪ names only the element itself and holds no ancestor inside
↪ it.
      - At least TWO descendants must agree on the prefix AND
↪ differ after it. One descendant alone cannot show where
↪ its code stops naming ancestors and starts naming itself.
      - Only WHOLE segments count. Never split a segment.
      - A descendant code that carries a structural LABEL
↪ word ("Foundation 1.7", "Benchmark 1.1", "Standard 2") is
↪ NOT a namespace path: the label names that descendant's OWN
↪ level, so the entire id after the label belongs to the
↪ descendant and none of it is an ancestor's code. Take a
↪ prefix only from bare dotted codes ("CD.WM.PK1", "I.A.2").
      This rule decides WHICH code an element gets. It NEVER
↪ licenses emitting an element you cannot see — rule 1 still
↪ governs that, and the heading line must literally appear in
↪ this chunk. If it does not, emit the descendants alone and
↪ take nothing from them.
   When (ii) or (iii) supplied the code, cite the line it came
↪ from in `source_text` IN ADDITION to the heading line —
↪ that line is part of the text you used (rule 7). Never
↪ instead of the heading line: rule 1's self-check still
↪ holds, and if a descendant's line is the ONLY line you can
↪ cite, you did not see the heading and must not emit the
↪ element at all.
   ONLY when all three come up empty — the document leaves
↪ this element genuinely uncoded — do you generate a stable
↪ ≤5-char uppercase abbreviation from the title, by this
↪ EXACT procedure:
   (a) Split the title into words on spaces and slashes. A
↪ hyphenated compound is ONE word ("self-selected" → one
↪ word contributing "S"; "Health/Mental Wellness" → three
↪ words).
   (b) DROP every CONNECTOR word. A connector is any of: `a an
↪ the and or but nor of to in on at by for from with into
↪ about over under through as`, and the symbol `&`.
↪ Connectors carry no meaning in a code — their initials only
↪ crowd out the words that do, and the code has just 5
↪ characters to spend. Keeping them yields a code made of
↪ function words: "Attends to an adult or peer who is
↪ communicating verbally or nonverbally" would give "ATAAO"
↪ ("Attends To An Adult Or") instead of "AAPWI", and "Engages
↪ in an activity for a sustained period of time" would give
↪ "EIAAF" instead of "EASPT". The code is the human-readable
↪ part of a standard's identifier, so it must spend its
↪ characters on the words that name the skill. This list is
↪ exact — do not extend it to other prepositions you may
↪ consider connector-like ("toward", "during", "between", "
↪ despite", "while"): those are CONTENT words for this
↪ purpose and their initials stay in the code. Match case-
↪ insensitively and ignore trailing punctuation.
   (c) If exactly ONE content word remains, the code is its
↪ first 4 letters ("Vocabulary" → "VOCA", "Initiative" → "
↪ INIT"). Otherwise the code is the first letter of each
↪ remaining content word, in order, capped at the first 5 ("
↪ Physical Development" → "PD", "Approaches to Learning" →
↪ "AL", "Concepts About Print" → "CP", "Social & Emotional
↪ Development" → "SED", "Language and Early Literacy" → "
↪ LEL", "Engages in an activity for a sustained period of
↪ time" → "EASPT").
   (d) If dropping connectors would leave NO words at all,
↪ keep the title's words as they are and apply (c) to those.
   Use the SAME code every time the same element appears.
   - When a heading is written as `<Label> <id>: <Title>` —
↪ where `<Label>` is ANY structural word the document uses to
↪ name a level (e.g. Strand, Concept, Sub-Strand, Goal,
↪ Pillar, Unit, Theme, Section, Element, Standard, Domain,
↪ Benchmark, Component, Cluster, Foundation, Outcome, or
↪ whatever else this particular document uses) and `<id>` is
↪ the position identifier at that level (a number "1", a
↪ letter "A", a token "1.2") — the label-plus-id together is
↪ the element's `code`, written as the label, ONE space, then
↪ the id (e.g. "Strand 1", "Concept 2", "Goal A", "Pillar
↪ 1.2"). Whatever punctuation the heading used to separate
↪ the label from the id — a colon, hyphen, en/em dash — is
↪ layout and does NOT enter the code: "Strand: 1.0 —
↪ Listening and Speaking" gives code "Strand 1.0", never "
↪ Strand: 1.0". The `title` is ONLY the text after the colon
↪ (e.g. "Strand 1: Self-Awareness and Emotional Skills" →
↪ title "Self-Awareness and Emotional Skills"). This rule
↪ applies to EVERY structural-label heading word, not just
↪ the examples — never leave the `<Label> <id>:` prefix
↪ inside `title`, regardless of which word the document chose
↪ for `<Label>`.
   - A structural label word contributes NOTHING to the code
↪ on its own. It earns a place in the code only through the
↪ position identifier attached to it: "Strand 1" works as a
↪ code because "1" says WHICH strand. So when a heading names
↪ the level but supplies no position identifier — `<Label><
↪ separator><Title>` with nothing between the label and the
↪ title, whatever the separator (colon, hyphen, en/em dash,
↪ or just a space): e.g. "Sub-Strand — Vocabulary", "Concept
↪ - Curiosity and Interest", "Domain: Number Sense", "Goal —
↪ Working Memory" — the label is pure level-naming, exactly
↪ like the trailing structural noun below. Drop it and derive
↪ the code from the TITLE ALONE using the ≤5-char
↪ abbreviation rule ("Vocabulary" → "VOCA", "Curiosity and
↪ Interest" → "CI", "Number Sense" → "NS", "Working Memory
↪ " → "WM") — unless the document supplies a code for the
↪ element somewhere else, in a caption or through its
↪ descendants' codes, in which case that recovered code wins
↪ over any abbreviation ((ii)/(iii) above). Never abbreviate
↪ the label into the code and never prefix the code with it
↪ ("Sub-Strand — Vocabulary" → "VOCA", NOT "SS-V"; "Concept
↪ - Curiosity and Interest" → "CI", NOT "C-CI"), and never
↪ use the raw heading line as the code.
   - The same principle holds when the structural label
↪ appears as a TRAILING noun on the heading instead of a
↪ prefix: a heading like `<Title> <Label>` (e.g. "Social and
↪ Emotional Development Domain", "Language and Literacy
↪ Standard", "Physical Development Area", "Number Sense
↪ Strand") names a `<Title>` of a level the document calls `<
↪ Label>`. The structural noun is the level word, NOT part of
↪ the name — emit only the bare `<Title>` ("Social and
↪ Emotional Development", "Language and Literacy", "Physical
↪ Development", "Number Sense"). This applies regardless of
↪ which structural noun the document uses (Domain, Standard,
↪ Area, Strand, Section, Component, Cluster, Foundation, etc
↪ .). The same-named element may also appear elsewhere on the
↪ page as a running page header in ALL CAPS (e.g. "SOCIAL
↪ EMOTIONAL DEVELOPMENT STANDARD" repeated at the top of
↪ every page) — that running header is page typography, not a
↪ separate element and not a code; if you emit anything for
↪ it, normalize it back to the same bare title-cased `<Title
↪ >` so the two variants share the SAME `code` and the SAME
↪ `title`. The abbreviation rule above ("≤5-char uppercase
↪ abbreviation from the title") still applies WHEN the
↪ document leaves the element uncoded: derive the code from
↪ the BARE title (e.g. "Social Emotional Development" → "
↪ SED"), NEVER use the heading text itself or the structural

```
↪ noun as the code. If the document DOES code it — in a
↪ caption, or through the shared prefix of its descendants'
↪ codes — that recovered code wins ((ii)/(iii) above).
   - When a lettered list IS the leaf indicators (per rule 2),
↪  the `code` is JUST that item's letter, lowercased ("a", "b
↪ ", "c", . . . ) — exactly as the document orders them,
↪ regardless of any OCR casing. Do NOT prepend the parent
↪ strand/concept number (no "S1C1a"), and do NOT uppercase it
↪  (no bare "C"). The downstream parser supplies the parent's
↪  number; the detector only needs the consistent local
↪ letter.
```

A.2.5 Confidence

This rule asks the model for a confidence band, but nothing downstream uses it: there is no threshold, no review gate, and no field marking an element for attention. It is reproduced so the bands can be compared with the scores the model actually emits (§5.5, §6.5).

```
5. `confidence`: 0.95+ if the depth map clearly applies;
↪ 0.80-0.94 if the chunk is ambiguous but the answer is
↪ likely; <0.70 if you are guessing.
```

A.3 Parsing

The parsing prompt states the output schema, then a list of rules governing how an indicator's fully-qualified code is composed from its chain of ancestors. The four reproduced below are the ones §5.3 draws on; the rest are in `anc/prompts.txt`.

A.3.1 Already-qualified codes are never re-prefixed

Applies at every level, not only the leaf. Without it a code the detector already resolved against its domain is composed onto its own domain a second time.

```
- ALREADY-QUALIFIED codes are used AS-IS and are NEVER re-
↪ prefixed — at EVERY level, not just the indicator. An
↪ element's detected `code` is already qualified when it is a
↪  dotted path whose FIRST segment is its own domain's code (
↪ after stripping any leading column token, below): an
↪ indicator detected as "SED.1.1.a" under domain "SED", or a
↪ sub_strand detected as "AB.CD" under domain "AB". For such
↪ an element, building the cumulative chain means CONFIRMING
↪ that prefix, not prepending it a second time: a sub_strand
↪ detected "AB.CD" under domain "AB" and strand "AB.2" →
↪ sub_strand_code "AB.CD" — NEVER "AB.2.AB.CD", and NEVER "AB
↪ .2.CD". Only an element whose code is a BARE LOCAL segment
↪ ("1.2", "A", "INIT", "Concept 1") gets its parents' code
↪ prepended.
```

A.3.2 A printed namespace may skip a level the hierarchy has

The Nevada case of §5.3, stated as a general principle: peeling ancestors off a descendant's code stops where the printed namespace stops, and the level outside it keeps the identifier its own heading supplied.

```
- A document's printed code NAMESPACE may SKIP a level that
↪ its hierarchy HAS. When you derive parent codes by peeling
↪ segments off a fully-qualified code, peel only as far as
↪ the namespace actually reaches: if peeling one more segment
↪  would give a level the SAME code as its own parent, that
↪ level is not in the namespace, and you must build it from
↪ its OWN heading's identifier appended to its parent's code
↪ instead. Worked example — indicators "AB.CD.PK1"/"AB.CD.PK2
↪ ", a sub_strand detected "AB.CD", a strand whose detected
↪ code is "<Something> Standard 2", a domain "AB": peeling
↪ gives sub_strand_code "AB.CD" (right), but peeling again
↪ would give the strand "AB", which is already the domain's
↪ code — so the strand instead takes its heading's bare
↪ identifier "2" → strand_code "AB.2". The levels the
↪ namespace DOES cover keep the document's spelling exactly:
↪ sub_strand_code "AB.CD", indicator_code "AB.CD.PK1". The
↪ sub_strand's code then does not literally extend the strand
↪ 's, and that is the correct answer: the identifier the
↪ document PRINTS is the one the standard is cited by, and it
↪  outranks cosmetic continuity of the chain.
```

A.3.3 Converting a label-and-identifier code

The detector correctly emits a heading's label-and-id as its code (`Benchmark 1.1`); converting that into the domain-qualified form is parsing's job. The rule is keyed on the shape `<Label> <id>` for any label word, with no list of label words anywhere in it.

```
- When an element's `code` is itself a structural label +
↪ identifier (e.g. "Strand 1", "Concept 1", "Goal 2", "Pillar
↪  A", "Unit 3" — any structural word the document uses,
↪ followed by a number or letter), use ONLY the bare
↪ identifier as that element's segment in the cumulative
↪ chain: "Strand 1" → segment "1", "Concept 1" → segment
↪ "1", "Pillar A" → segment "A". The label word merely
↪ names the level and is already captured by the element's `
↪ level`; it must NOT appear inside the cumulative `code`.
↪ Example: a strand with code "Strand 1" under domain "SED"
↪ → strand_code "SED.1" (not "SED.Strand 1"); a sub_strand
↪ with code "Concept 1" under that strand → sub_strand_code
↪  "SED.1.1" (not "SED.Strand 1.Concept 1"). Apply this to
↪ ANY label word, not just the examples.
```

A.3.4 Disambiguating side-by-side columns

Column indicators share a code stem by construction, so the identifier must carry the distinction itself. Age-range columns and proficiency columns take different suffixes, and an indicator that came from no column takes none.

```
- DISAMBIGUATE side-by-side columns by APPENDING a token to
↪ the END of the indicator code, so two variants of the same
↪ outcome (which share an identical base code) get DISTINCT
↪ indicator codes — and therefore distinct standard_ids.
↪ Append the token to the INDICATOR code ONLY; NEVER add it
↪ to the domain, strand, or sub_strand code. Choose the token
↪  by the column's type:
  - AGE-RANGE column (the cell carries an age range — e.g. "
↪ Early (3 to 4 ½ Years)", "Later (4 to 5 ½ Years)", "PK3",
↪  "PK4"): append the SAME normalized month range you put in
↪ this indicator's `age_band`, written exactly as "{start}-{
↪ end}" (no spaces, no "months"). Examples: a PK3 outcome
↪ with base `I.A.2` → `I.A.2.36-48`, its PK4 variant → `I.
↪ A.2.48-60`; an "Early" outcome with base `AL.1.0.INIT.1.2`
↪  → `AL.1.0.INIT.1.2.36-54`, its "Later" variant → `AL
↪ .1.0.INIT.1.2.48-66`. Apply this even when only one age
↪ column is present for the outcome (a lone PK4 outcome `VI.A
↪ .1` still becomes `VI.A.1.48-60`).
  - NON-AGE column (a proficiency or similar label where the
↪ columns share one age range, so `age_band` is null — e.g. "
↪ Discovering"/"Developing"/"Broadening"): append the FIRST
↪ FOUR LETTERS of the column label, UPPERCASED. Examples:
↪ base `ELD.1.0.VOCA.1.1` → `ELD.1.0.VOCA.1.1.DISC` (
↪ Discovering), `ELD.1.0.VOCA.1.1.DEVE` (Developing), `ELD
↪ .1.0.VOCA.1.1.BROA` (Broadening).
  - If the indicator does NOT come from an age/column cell (no
↪  per-column `age_band` and no `column_label`), append
↪ NOTHING — leave the base code as-is.
```

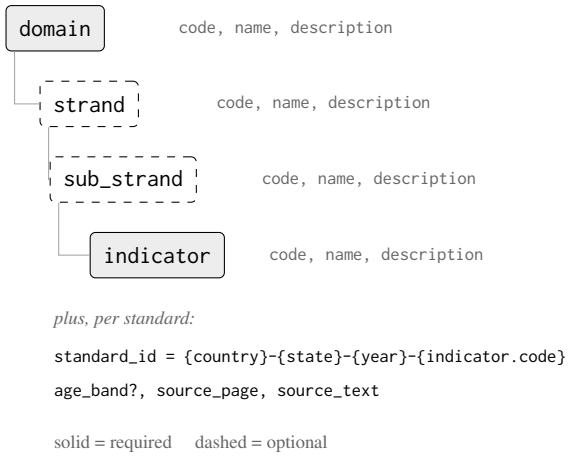


Figure 2: The canonical schema. Four hierarchy levels, of which `strand` and `sub_strand` are optional; each carries a code, a name and an optional description. A stored record is one indicator together with its fully resolved ancestry, not a node in a tree (§4).

B Canonical Schema

Figure 2 gives the canonical schema of §4 as a tree. Field names are those of the implementation's typed model, which every parsed record is validated against before persistence; the per-element human verification state (`human_verified`, `verified_at`, `verified_by`) is stored alongside each level and is omitted from the figure because it is editorial state rather than extracted content.

C Representative Golden Examples

Both suites are graded against independently annotated golden sets with different shapes: the detector golden is a flat list of elements, the parser golden a nested record per standard. The examples below were each chosen to show one convention the body of the paper turns on. Free-text descriptions are elided for length; codes, levels and identifiers are reproduced in full.

KY: a `<Label> <id>` heading, where the label-and-id *is* the code

`level`	`sub_strand`
`code`	`Benchmark 1.1`
`title`	Maintains focus and sustains attention.
`description`	*null*
`source_page`	2

Golden entry `KY-SUB-01`, `evaluation/ground_truth_detector/KY.json`.

KY: the same element after parsing, its label form resolved to a domain-qualified code

`standard_id`	`US-KY-2021-AL.1.1.EASPT`
`domain.code`	`AL`
`domain.name`	Approaches to Learning
`strand.code`	`AL.1`
`strand.name`	Sustains attention and persists with challenging activities and experiences.
`sub_strand.code`	`AL.1.1`
`sub_strand.name`	Maintains focus and sustains attention.
`indicator.code`	`AL.1.1.EASPT`
`indicator.name`	Engages in an activity for a sustained period of time.
`age_band`	36-60

Golden entry `KY-STD-01`, `evaluation/ground_truth_parser/KY.json`. Descriptions elided.

NV: a printed namespace that skips a level: the sub-strand code does *not* extend the strand's

`standard_id`	`US-NV-2023-SS.ID.PK1`
`domain.code`	`SS`
`domain.name`	Social Studies
`strand.code`	`SS.1`
`strand.name`	Demonstrate a basic awareness of self as an individual, within the context of a group and community.
`sub_strand.code`	`SS.ID`
`sub_strand.name`	Individual Development and Cultural Identity
`indicator.code`	`SS.ID.PK1`
`indicator.name`	Identify and describe characteristics of self that are unique from others.
`age_band`	36-60

Golden entry `NV-STD-01`, `evaluation/ground_truth_parser/NV.json`. Descriptions elided.

CO: a three-level document, where `sub_strand` is absent by design, not missing

`standard_id`	`US-CO-2020-PDH.1.1`
`domain.code`	`PDH`
`domain.name`	Physical Development & Health
`strand.code`	`PDH.1`
`strand.name`	Health, Safety and Nutrition
`sub_strand`	*absent; this document has no such level*
`indicator.code`	`PDH.1.1`
`indicator.name`	Possess good overall health, including oral, visual, and auditory health, and be free from communicable or pre [...]
`age_band`	36-60

Golden entry `CO-IND-01`, `evaluation/ground_truth_parser/CO.json`. Descriptions elided.

CA: a side-by-side column variant, disambiguated inside the indicator code

`standard_id`	`US-CA-2021-AL.1.0.INIT.1.2.36-54`
`domain.code`	`AL`
`domain.name`	Approaches to Learning
`strand.code`	`AL.1.0`
`strand.name`	Motivation to Learn
`sub_strand.code`	`AL.1.0.INIT`
`sub_strand.name`	Initiative
`indicator.code`	`AL.1.0.INIT.1.2.36-54`
`indicator.name`	Initiative
`age_band`	36-54

Golden entry `CA-IND-01-EARLY`, `evaluation/ground_truth_parser/CA.json`. Descriptions elided.

D Confidence Score Distribution

Table 9 gives the detector's confidence distribution by state and by hierarchy level, read by §6.5, together with two observations cut from that section. They are here because the score gates nothing in the pipeline and the section's conclusion, that the distribution is too degenerate to threshold on, is carried by the prose.

Run-to-run drift. The score is not stable between runs either. On an earlier freeze of the same corpus under the same configuration, 89.7% of indicators scored 0.95 or above, against 93.5% in the recorded run: a four-point swing on an unchanged corpus, wider than most of the distinctions a review threshold would be asked to draw.

The lowest-scoring element. The single confirmed hallucination in the corpus is also its lowest-scoring element, the only one at 0.90, and it holds that position across recordings. This is not evidence of a working fabrication detector. There is exactly one invented element in the audited corpus, so the false-negative rate of any cut is unmeasurable; the threshold the prompt defines would not catch it; and all 39 real-but-split titles, produced by column interleaving rather than by the model, score at or above 0.95.

	n	Range	≥ 0.95
By hierarchy level			
domain	16	0.95–0.97	16/16
strand	51	0.92–0.97	48/51
sub-strand	49	0.92–0.97	47/49
indicator	263	0.90–0.97	246/263
By human false-positive audit verdict			
matched a golden	135	0.92–0.97	117/135
real, unannotated	117	0.92–0.97	115/117
real, split title	39	0.95–0.96	39/39
real, reprinted	5	0.95–0.95	5/5
invented	1	0.90–0.90	0/1
All	379	0.90–0.97	357/379

Table 9: Detector self-reported confidence, re-measured over the 379 elements of the recorded run. The prompt asks for ≥ 0.95 when the depth map clearly applies, 0.80–0.94 when the chunk is ambiguous, and < 0.70 when guessing; the score in fact takes 5 distinct values in $[0.90, 0.97]$ and uses the bottom band never. **Nothing in the pipeline thresholds this score** (§6.5). *_only_subset* tier. Source: `task8_20260904`.

E Corpus

Table 10 identifies every source document by publisher, year and page count at each tier, and Table 11 gives the pages of each published PDF that the `_trimmed` and `_only_subset` tiers retain, in the published document's own page numbering, so that the trimming can be reproduced exactly from the issuing agency's copy (§3).

F Deployed Architecture

Figure 3 shows the deployed system. Documents flow through the six pipeline stages of §5, numbered 1–8 in the figure because the two batched stages each occupy two positions (prepare, then map and merge) and are realized as ten Lambda handlers in all. Stages communicate by writing JSON to S3 rather than by passing payloads directly, so each stage's input and output can be inspected in isolation when a run is debugged.

Detection and parsing are batched through a prepare $\rightarrow$ Step Functions `Map` $\rightarrow$ merge pattern, with at most three concurrent workers, so that a long document does not exhaust the Lambda timeout. Detection batches by text-block chunk and parsing by domain, which keeps related elements in the same call. Detection is itself two-pass: the depth-map pass infers the document's nesting hierarchy before any element is classified against it.

Figure 4 shows the end of that pipeline as a user meets it: extracted standards resolved into the canonical schema and served through the Standards Explorer, where the per-element verification workflow of §5.5 is carried out.

State	Role	Year	Publisher	Full	Trim.	Subset
CA	dev	2021	CA Dept. of Education	68	–	13
AZ	dev	2018	AZ Dept. of Education	217	65	15
TX	dev	2022	TX Education Agency	87	71	9
CO	dev	2020	CO Dept. of Early Childhood	41	41	10
NV	validation	2023	NV Dept. of Education	98	71	15
KY	validation	2021	Governor's Office of Early Childhood	120	52	8

Table 10: Corpus provenance. *Full* is the published PDF; *Trim.* removes non-standards matter while retaining every standard; *Subset* is the one-to-two-domain excerpt the headline quality measurements use, and is a genuine reduction in coverage. Colorado's *Full* is its 41-page Ages 3–5 publication (§3). These are third-party state-agency documents, identified here rather than redistributed (§10). Page counts verified with PyMuPDF. Source: `standards_tracking.md`, `corpus_tiers.json`.

State	Tier	pp	Retained, by PDF page index	Same pages, as printed
AZ	`_trimmed`	65	2, 21, 23–28, 41, 43–50, 66, 70–82, 107, 110–120, 139, 142–145, 156, 158–162, 179, 181–185, 198, 201–204 (of 217)	18, 20–25, 38, 40–47, 63, 67–79, 104, 107–117, 136, 139–142, 153, 155–159, 176, 178–182, 195, 198–201
AZ	`_only_subset`	15	2, 21, 23, 66, 70–78, 81–82 (of 217)	18, 20, 63, 67–75, 78–79
CA	`_only_subset`	13	1–5, 11–13, 18–22 (of 68)	*not recovered*
CO	`_trimmed`	41	1–41 (of 41)	*not recovered*
CO	`_only_subset`	10	1, 3–9, 11–12 (of 41)	*not recovered*
TX	`_trimmed`	71	1, 9–65, 69–81 (of 87)	*not recovered*
TX	`_only_subset`	9	1, 9–13, 59–61 (of 87)	*not recovered*
NV	`_trimmed`	71	1, 3, 16–84 (of 98)	*not recovered*
NV	`_only_subset`	15	1, 3, 23–26, 54–57, 61–65 (of 98)	*not recovered*
KY	`_trimmed`	52	1, 52–102 (of 120)	39–89
KY	`_only_subset`	8	1, 52–55, 65, 71–72 (of 120)	39–42, 52, 58–59

Table 11: Pages retained at each corpus tier, so the hand trimming can be reproduced from the issuing agency's copy. **Two numberings are given and they are not interchangeable**: the PDF page index counts pages in the file, while the printed number is what appears on the page, and they differ by each document's unnumbered front matter (Kentucky's PDF page 52 is printed page 39). Navigate a file by the first and a paper copy by the second. Ranges were recovered by matching each tier page's text back to the published PDF; no annotation or model call is involved. The printed column is populated only where page-number extraction is reliable across the tier, and left blank rather than guessed otherwise. Every tier page matched a published page. California has no `_trimmed` tier and Colorado's is the whole document. Source: `corpus_page_ranges.json`.

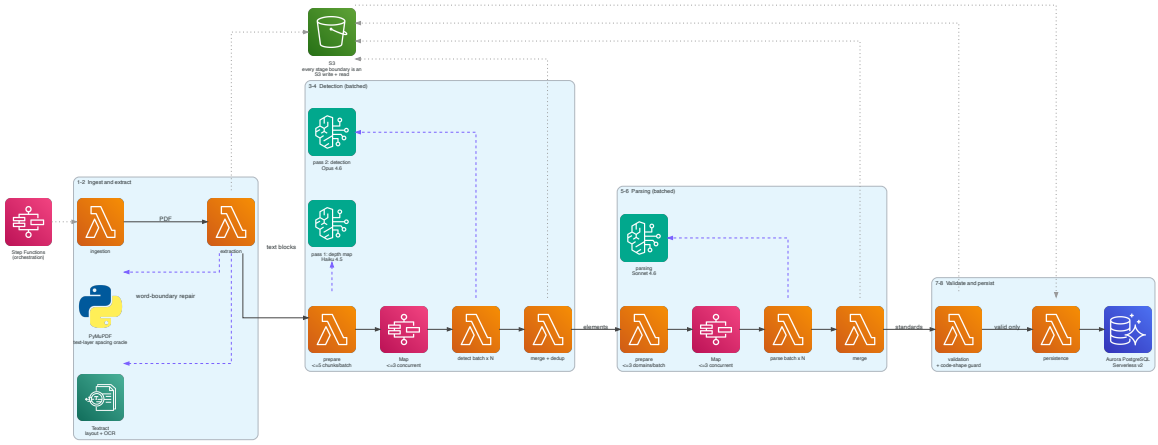


Figure 3: Pipeline architecture. Solid arrows carry data, dashed arrows are model calls, and dotted arrows mark JSON exchanged through S3. Regenerate with `python paper/analysis/make_architecture_figure.py`.

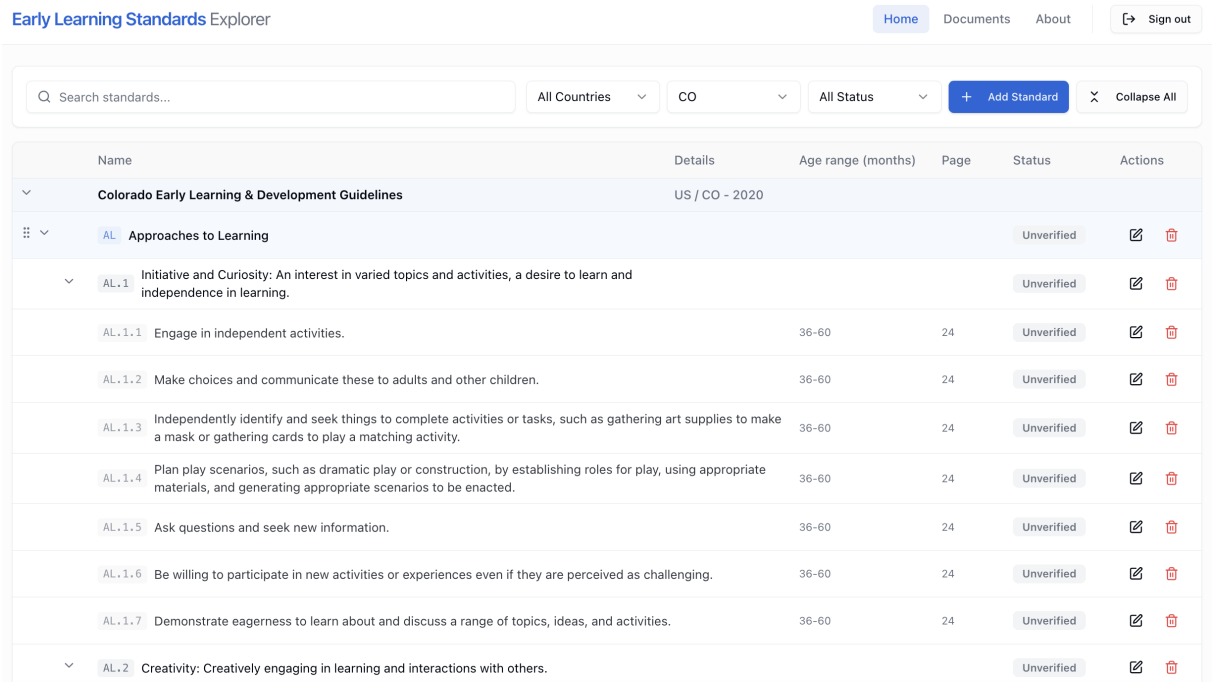


Figure 4: The Standards Explorer, showing extracted Colorado standards resolved into the canonical schema of §4: domain AL, strand AL.1, and the indicators beneath it, each row carrying its age range and source page. The *Status* column is the per-element human verification state of §5.5, which is **independent of the model's confidence score**.